

ThreatFusion AI

Capstone Report

A defensive cyber threat intelligence fusion pipeline: end-to-end execution over a 60-report corpus, an honest evaluation of the ATT&CK classifier, and the defensive products derived from it.

Name	Asim Shayan Khan
Programme	AI-Driven Cybersecurity and Digital Forensics Fellowship
Deliverable	Capstone Report (AICS-107 Capstone Report Template, 12 sections + Appendix A)
Pipeline run	17 July 2026, 16:42:12Z · 60 cases · full run-pipeline
Report date	18 July 2026
Environment	Google Colab · Python 3.12.13 · CPU runtime · project resident in Google Drive
Data	Synthetic training corpus only. No live feeds queried.
Classification	TLP:CLEAR

SAFETY AND SCOPE STATEMENT

This is a defensive training exercise. Every indicator in this report is synthetic by construction: IPv4 values are drawn from the RFC 5737 documentation ranges (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24), domains use reserved or example namespaces (.invalid, .test, example.com/.net/.org), and file hashes are placeholder values consisting of a single repeated digit. None of them resolve and none of them are hostile. No URL or domain was browsed. No sample was retrieved by hash. No live threat feed was queried — the optional Lab 8 collectors were not authorised for this run and were not executed. No API key existed in the environment at any point. Every indicator is defanged in the body of this report as a matter of discipline, not necessity.

Every ATT&CK mapping produced by the classifier in this project is an AI suggestion requiring analyst validation. Section 6 demonstrates why that caveat is load-bearing rather than boilerplate, and the detection backlog in Section 11 is built from analyst-validated labels rather than model output as a direct consequence.

Contents

1. Executive Summary	2
<i>The run · the headline finding · why the 0.90 is an artefact</i>	
2. Intelligence Requirements	4
<i>PIR-1 · PIR-2 · PIR-3 · scope and constraints</i>	
3. Data Sources and Handling	5
<i>Source matrix · record structure · the indicator set · handling controls</i>	
4. System Architecture	8
<i>Module data flow · execution environment · the timezone patch · end-to-end run</i>	
5. AI Model Design	10
<i>Task formulation · design decisions and verdicts · why not a larger model</i>	
6. Evaluation Results	11
<i>Reported metrics · the sampling artefact · what the model learned · leave-one-template-out · novel phrasing · improvement plan</i>	
7. ATT&CK Mapping	15
<i>Techniques evidenced · co-occurrence · provenance and confidence · risk scoring · the Lab 6 no-op</i>	
8. Knowledge Graph and Campaign Analysis	18
<i>Graph composition · the clustering result · why it is an artefact · no campaign declared</i>	
9. STIX, MISP and OpenCTI Sharing	20
<i>The bundle as exported · two defects · platform mapping · the decision not to share</i>	
10. Analyst Reports	22
<i>Reports produced · assessment of the product · executive version</i>	
11. Defensive Recommendations	24
<i>Detection backlog · prioritised recommendations · what the pipeline is good for</i>	
12. Limitations and Future Work	26
<i>Limitations · semantic matching over supervised training · future work · closing assessment</i>	
A. Appendix A · Evidence Portfolio Index	29
<i>Submission checklist · open items · figure index · full case ledger</i>	

1. Executive Summary

ThreatFusion AI is a defensive cyber threat intelligence fusion pipeline. It ingests CTI reporting, extracts indicators of compromise, proposes MITRE ATT&CK technique mappings with a supervised classifier, assigns a risk score, assembles a knowledge graph, exports a STIX 2.1 bundle, and writes analyst-ready reports. This document reports a complete end-to-end execution over the 60-report training corpus, run on 17 July 2026 in Google Colab under Python 3.12.13 with the project resident in Google Drive.

The pipeline works as designed. A single `run-pipeline` invocation processed all 60 cases in ten seconds and produced every required artefact: an 85-node, 302-edge knowledge graph; a 362-object STIX 2.1 bundle; five Markdown analyst reports; and a campaign clustering pass. A five-ticket detection engineering backlog was derived from it. Every submission-checklist item is present.

The headline finding is negative, and it is the most valuable result in the project. The ATT&CK classifier reports a macro F1 of **0.9000** on the shipped evaluation split and misclassifies exactly zero of the 150 test label cells. That number is an illusion. Three independent diagnostics establish it:

1. The corpus contains 60 reports but only **five distinct label combinations**, each one fixed by the report template that produced it. The task the model actually solves is five-way template identification, not ten-technique multi-label mapping.
2. `features.py` vectorises `title + ' ' + text`, and the title carries the template name. The model's highest-weighted tokens for T1027, T1053 and T1566 include literal metadata strings — `'bravo defensive'`, `'alpha defensive'`. It learned *which template it was looking at*, then recited that template's labels.
3. Held out an entire template family and tested on it, micro F1 collapses from **0.900 to a mean of 0.273**. Given five plainly-worded analyst sentences containing no template keywords, the model scored **0 out of 5**.

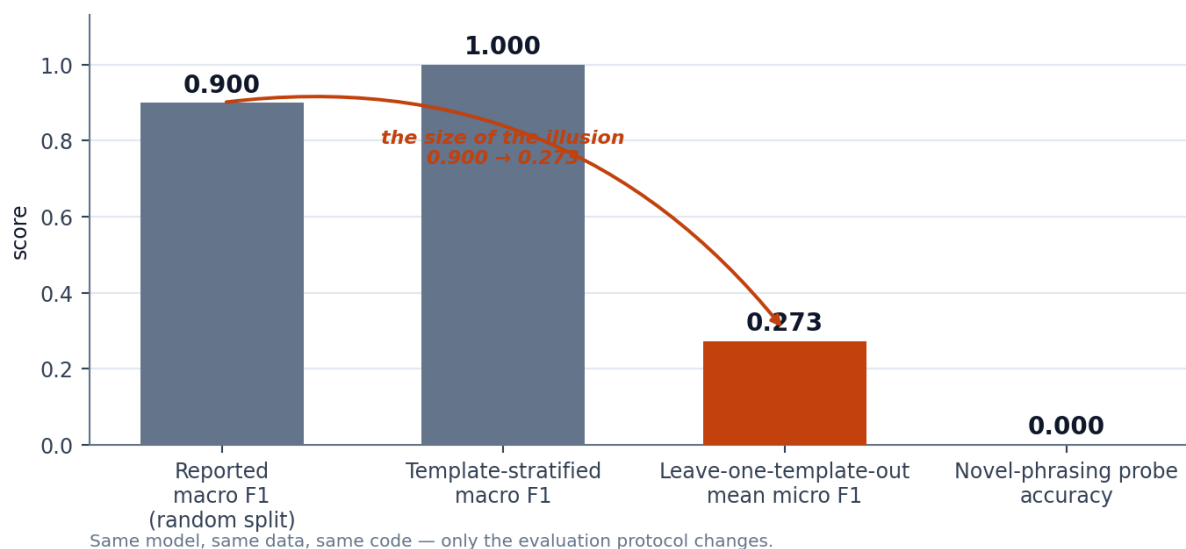


Figure 1. *The size of the illusion. Same model, same weights, same corpus — only the evaluation protocol changes. The 0.900 headline and the 0.273 reality are the same classifier measured honestly and dishonestly.*

A zero-shot embedding recommender built in Lab 10 (`all-MiniLM-L6-v2`, never trained on this corpus) placed a correct technique first on all five sampled cases. That contrast is the central evidence of this report: an off-the-shelf semantic model that has never seen the data outperforms a classifier trained on it, because the classifier learned the filenames rather than the tradecraft. The 0.900 is not a result to defend — it is a result to explain, and Section 6 explains it.

Three further defects were identified and are documented with evidence. The STIX bundle contains 362 objects but only 85 distinct identifiers and carries zero relationships, so an importing platform would

receive 60 disconnected reports and a pile of orphaned indicators (Section 9). The single "campaign cluster" spanning all 60 cases is a graph artefact of including ATT&CK technique nodes as hubs, not evidence of a shared actor (Section 8). And the Lab 6 risk-tuning patch produced byte-identical scores before and after it was applied, which means the patch silently failed (Section 7).

Operational bottom line. No mapping produced by the model in this run may drive a defensive decision without analyst validation. That constraint was applied rather than merely stated: the Lab 11 detection backlog is built from analyst-validated labels, not model output, and all five tickets ship as **DRAFT**. Where the pipeline could not answer an intelligence question honestly — specifically the campaign attribution question — this report records that it could not, rather than reporting the artefact as a finding.

Table 1. *The run at a glance. Every figure below is traceable to a named artefact in the evidence portfolio (Appendix A).*

Dimension	Result	Where
Cases processed	60 (5 template families: ALPHA 12, BRAVO 17, CHARLIE 14, DELTA 9, ECHO 8)	\$3, \$7
Runtime	10 s, CPU runtime, no GPU required	\$4
IOCs extracted	72 instances · 15 unique (5 domain, 5 IPv4, 5 SHA-256)	\$3
ATT&CK techniques	10 modelled, all 10 evidenced in the corpus	\$7
Reported macro F1	0.9000 — an artefact, see \$6	\$6
Honest generalisation	0.273 mean micro F1 on unseen templates · 0/5 on novel phrasing	\$6
Knowledge graph	85 nodes · 302 edges (230 uses-technique, 72 has-ioc)	\$8
Campaign clusters	1 cluster of 60 cases — rejected as an artefact	\$8
STIX bundle	362 objects · 85 distinct ids · 0 relationships	\$9
Analyst reports	5 generated (requirement: ≥3)	\$10
Detection backlog	5 DRAFT tickets from validated mappings	\$11
Severity distribution	51 critical · 9 high · 0 medium · 0 low	\$7

2. Intelligence Requirements

Intelligence that answers no question is trivia. Three Priority Intelligence Requirements were declared for this run, each tied to a decision a defender actually has to make, an owner who has to make it, and a named pipeline output that answers it. Every recommendation in Section 11 traces back to one of them.

2.1 Priority Intelligence Requirements

Table 2. PIRs declared for this run, with the decision each supports and the pipeline output that answers it.

PIR	Question	Decision it supports	Answered by	Status
PIR-1	Which ATT&CK techniques appear across our reporting, and which are frequent enough to justify detection investment this quarter?	Detection engineering backlog prioritisation; SOC content roadmap	Technique frequency across the corpus (§7); Lab 11 backlog (§11)	ANSWERED
PIR-2	Which cases show ransomware-stage impact (T1486) or credential access (T1003), so that incident response and backup assurance can be pre-positioned?	IR readiness; backup restore verification; credential hygiene programme	Risk scoring and severity banding (§7); analyst reports (§10)	ANSWERED
PIR-3	Do the reported cases share infrastructure to a degree that justifies treating them as one campaign rather than independent incidents?	Open one campaign investigation vs. many; estate-wide indicator blocking	Knowledge graph and clustering (§8)	NOT ANSWERABLE

PIR-1 — answered. Ten techniques are evidenced across the corpus. Five clear the evidence threshold set for this run (≥ 25 of 60 cases): T1059 (38), T1041 (34), T1027 (26), T1071 (26) and T1003 (25). Those five drove the detection backlog. The ranking must be read with the caveat in Section 12: technique frequency here is a property of the generator's template mix, not of any real threat landscape.

PIR-2 — answered. Fourteen of 60 cases (23%) carry T1486 Data Encrypted for Impact and all fourteen score critical at 95.7. Twenty-five of 60 (42%) carry T1003 OS Credential Dumping, peaking at 97.2. Both feed concrete recommendations in Section 11. The answer carries a qualifier: because 51 of 60 cases land in the critical band, severity ranks almost nothing, so the *selection* of these cases is sound but their *ordering* by risk score is not (§7.4).

PIR-3 — not answerable, and that is the correct answer. The clustering step returns a single cluster containing all 60 cases. Section 8 shows this is a structural consequence of including ATT&CK technique nodes in a connected-components calculation, not a finding about shared infrastructure. Re-running the analysis on the case↔indicator subgraph alone yields seven components rather than one — but even that result is driven by the corpus recycling only 15 indicators across 60 cases. Neither projection supports a campaign conclusion. **This report therefore declares no campaign.** Recording an unanswerable PIR as unanswered is an intelligence outcome; reporting the artefact as a campaign would have been a fabrication.

2.2 Scope and constraints

- **Sector assumption:** enterprise Windows estate with a financial-services weighting, which is the assumption the Lab 6 risk-scoring tuning was written against.
- **Data scope:** the 60-report synthetic training corpus only. No live feed was queried; the optional Lab 8 collectors were not authorised for this run and were not executed.
- **Technique scope:** 10 ATT&CK Enterprise techniques. Anything outside that set is invisible to this pipeline rather than merely misclassified — see Section 12.
- **Decision scope:** this run supports detection engineering and IR readiness planning. It supports no attribution, no blocking, and no external sharing.

for `v` in `iocs.values()` if `isinstance(iocs, dict)` else `0`, but `iocs` is a JSON *list* of indicator objects, not a dict — so the `isinstance` guard fails and the expression silently returns `0`. Extraction in fact worked: 72 indicator instances are present in the graph and the STIX bundle. Corrected expression: `len(iocs) if isinstance(iocs, list) else 0`.

Why it is in this report: a silent zero that looks like a plausible result is exactly the class of bug that reaches a briefing slide unchallenged. It was caught by cross-checking the DataFrame against the graph edge count, which is the general defence.

3.3 The indicator set

Fifteen unique indicators appear across the corpus, every one of them non-routable by construction. All are defanged below as a matter of discipline. The right-hand column is the number of cases each indicator touches — that concentration is what Section 8 has to reckon with when it evaluates the campaign clustering.

Table 4. The complete indicator set: 15 unique indicators, 72 case-linkages. Every value is drawn from a reserved range that cannot resolve.

Indicator (defanged)	Type	Why it is safe	Cases
update-example[.]com	domain	example.com — IANA reserved for documentation	10
storage-example[.]invalid	domain	.invalid — RFC 2606 reserved TLD	9
cdn-example[.]net	domain	example.net — IANA reserved	7
203.0.113[.]42	ipv4	TEST-NET-3 — RFC 5737 documentation range	7
login-example[.]org	domain	example.org — IANA reserved	5
2222...2222	sha256	Placeholder — single repeated digit, no real sample	5
192.0.2[.]200	ipv4	TEST-NET-1 — RFC 5737	5
203.0.113[.]10	ipv4	TEST-NET-3 — RFC 5737	5
3333...3333	sha256	Placeholder	4
1111...1111	sha256	Placeholder	4
news-example[.]test	domain	.test — RFC 2606 reserved TLD	3
0000...0000	sha256	Placeholder	3
198.51.100[.]77	ipv4	TEST-NET-2 — RFC 5737	2
192.0.2[.]14	ipv4	TEST-NET-1 — RFC 5737	2
4444...4444	sha256	Placeholder	1

3.4 Handling controls applied

Table 5. Data-handling and safety controls, and the evidence that each was actually applied rather than merely asserted.

Control	How it was applied	Evidence
Offline-by-default	No live feed queried. <code>collectors.py</code> never imported or invoked.	No network cell in the notebook beyond <code>pip</code> and the Hugging Face model download for Lab 10
No indicator detonation	No URL browsed, no domain resolved, no sample fetched by hash.	All 15 indicators are non-resolvable by construction (§3.3)
Key hygiene	No API key was ever present in the environment. Nothing to leak, redact or accidentally commit.	<code>OTX_API_KEY</code> unset; Lab 8 skipped
Defanging	Every indicator is defanged in written output.	§3.3, §10
TLP marking	TLP:CLEAR asserted on every case and every analyst report.	Figures 2 and 30

Control	How it was applied	Evidence
Sharing guard	Every STIX indicator carries labels: ["training-do-not-touch-live"]. Every report SDO carries ["aics-107", "training", <severity>].	§9; verified across all 72 indicator objects
AI/analyst separation	Detection backlog built from validated labels, not model predictions. Every ticket marked DRAFT.	§11, Figures 33–34
Persistence	Project held in Google Drive so no work is lost to runtime recycling; no secrets in the notebook.	Figure 4

On the TLP:CLEAR marking. TLP:CLEAR is defensible here only because the data is fabricated and contains nothing sensitive about any real organisation. It is a property of *this corpus*, not of the pipeline. Real reporting produced by this same code would require TLP:AMBER or tighter, and the marking is currently hardcoded rather than derived — a case-by-case TLP decision is not something the pipeline is equipped to make. That is noted as an open item in Section 12.

4. System Architecture

ThreatFusion AI is a five-stage pipeline driven by a single CLI. The stages map onto the classical intelligence cycle: collection, processing, analysis, production and dissemination. Figure 3 shows the module graph as it actually executes, including one detail that a tidier diagram would hide — `sample_cti_reports.jsonl` is read twice, once by the training step and again at the top of the enrichment loop, rather than flowing strictly downward.

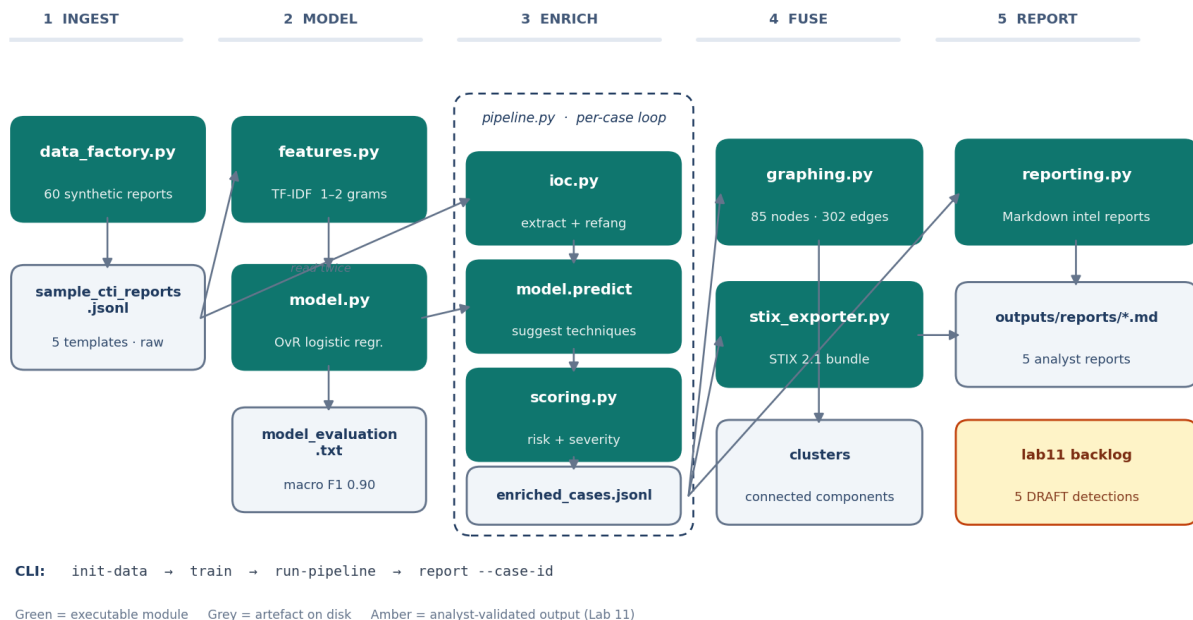


Figure 3. ThreatFusion AI module data flow. Green nodes are executable modules, grey nodes are artefacts on disk, and the amber node is the analyst-validated Lab 11 output that deliberately bypasses the model's predictions.

Table 6. Module responsibilities and the artefact each one produces.

Stage	Module	Responsibility	Artefact produced
1 · Ingest	<code>data_factory.py</code>	Generates the synthetic corpus from five report templates using only reserved indicator ranges	<code>data/raw/sample_cti_reports.jsonl</code> 60 reports
2 · Model	<code>features.py</code> <code>model.py</code>	TF-IDF vectorisation (1–2 grams) over title+text; one-vs-rest logistic regression over 10 technique labels	<code>models/outputs/model_evaluation.txt</code>
3 · Enrich	<code>pipeline.py</code> <code>ioc.py</code> <code>scoring.py</code>	Per-case loop: extract and refang indicators → suggest techniques → compute risk score and severity band	<code>data/processed/enriched_cases.jsonl</code>
4 · Fuse	<code>graphing.py</code> <code>stix_exporter.py</code>	Builds the NetworkX case↔technique↔IOC graph, computes connected-component clusters, and exports a STIX 2.1 bundle	<code>threat_knowledge_graph.graphml/.json</code> <code>threatfusion_bundle.json</code> <code>campaign_clusters.json</code>
5 · Report	<code>reporting.py</code>	Renders per-case Markdown intelligence reports with defensive actions	<code>outputs/reports/*_intel_report.md</code>

4.1 Execution environment

The course setup guide assumes an Ubuntu virtual machine with a Python virtual environment. This run used Google Colab instead. Colab is Ubuntu underneath, so every artefact is byte-for-byte the kind of output the rubric asks for, but two things change materially and both were handled explicitly:

- **No virtual environment.** Dependencies were installed into the base interpreter with `pip install -r requirements.txt` rather than through `setup_linux.sh`, which would have built a `.venv` needing reactivation in every cell.
- **No persistence.** Colab recycles `/content` on disconnect. The project was therefore held at `/content/drive/MyDrive/AICS107/ThreatFusionAI_Project` so that data and outputs survive a dropped runtime. This also means the submission needs no

The runtime was CPU-only (Figure 4), which is sufficient for Labs 1–9. Only the Lab 10 embedding extension benefits from a GPU, and it completed acceptably on CPU regardless.

```
[1]
✓ 0s !python --version
      !nvidia-smi || echo "CPU runtime (fine for Labs 1-9)"

... Python 3.12.13
/bin/bash: line 1: nvidia-smi: command not found
CPU runtime (fine for Labs 1-9)

[2]
✓ 24s from google.colab import drive
      drive.mount('/content/drive')

      import os
      WORK = '/content/drive/MyDrive/AICS107'
      os.makedirs(WORK, exist_ok=True)
      print("Working folder:", WORK)

Mounted at /content/drive
Working folder: /content/drive/MyDrive/AICS107

[3]
✓ 0s %cd "/content/drive/MyDrive/AICS107/ThreatFusionAI_Project"
      !ls

/content/drive/MyDrive/AICS107/ThreatFusionAI_Project
data      models    README.md run_full_pipeline.sh threatfusion_ai
graph.html outputs   requirements.txt setup_linux.sh
```

Figure 4. Environment verification and Drive mount. Python 3.12.13, CPU runtime confirmed, and the project root established in Drive so outputs survive runtime recycling.

4.2 One code change made before the first run

`stix_exporter.py` called `datetime.datetime.utcnow()`, which is deprecated from Python 3.12 and emits a `DeprecationWarning` on every export. Because this runtime is 3.12.13, the warning fires on every one of the 362 objects written. It was patched before the first pipeline run:

```
datetime.datetime.utcnow().replace(microsecond=0).isoformat()
→ datetime.datetime.now(datetime.timezone.utc).replace(microsecond=0,
tzinfo=None).isoformat()
```

The replacement is deliberately conservative. Dropping `tzinfo` after converting to UTC preserves the exact naive-UTC `...Z` string that the STIX writer appends, so the output format is unchanged while the deprecated call is gone. A less careful fix — leaving the timezone attached — would have produced `+00:00Z` suffixes and broken every timestamp in the bundle. Verified in the emitted objects: `'created': '2026-07-17T16:42:12Z'`.

```

[5] | pip install -q -r requirements.txt
[6] | import pathlib
    | p = pathlib.Path('threatfusion_ai/stix_exporter.py')
    | s = p.read_text()
    | s = s.replace(
    |     "datetime.datetime.utcnow().replace(microsecond=0).isoformat()",
    |     "datetime.datetime.now(datetime.timezone.utc).replace(microsecond=0, tzinfo=None).isoformat()")
    | p.write_text(s)
    | print("stix_exporter.py patched (timezone-aware timestamps)")
    |
    | stix_exporter.py patched (timezone-aware timestamps)
[7] | python -m threatfusion_ai.cli init-data
    | head -n 2 data/raw/sample_cti_reports.jsonl
    |
    | Data created:
    | /content/drive/MyDrive/AICS107/ThreatFusionAI_Project/data/raw/sample_cti_report
    | s.jsonl
    | {"id": "LAB-BRAVO-000", "source": "synthetic-training-corpus", "title": "LAB-BRAVO defensive CTI report 000", "text": "endpoint logs show obfuscated PowerShell-like command a
    | {"id": "LAB-CHARLIE-001", "source": "synthetic-training-corpus", "title": "LAB-CHARLIE defensive CTI report 001", "text": "ransomware-style impact indicators were discovered
  
```

Figure 5. Dependency install, the timezone-aware patch to `stix_exporter.py`, and corpus generation. The patch was applied before the first export so no artefact in this submission carries the deprecated call.

4.3 End-to-end execution

The full pipeline runs from one command. Ten seconds, 60 cases, every artefact written (Figure 6). Note 'clusters': 1 in the return value — that single number is the subject of Section 8.

```

[9] | python -m threatfusion_ai.cli run-pipeline
    | ls outputs outputs/graphs outputs/reports outputs/stix
    |
    | {
    |     'cases': 60,
    |     'graph':
    |       '/content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/graphs/threat_kno
    | wledge_graph.graphml',
    |     'stix':
    |       '/content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/stix/threatfusion
    | _bundle.json',
    |     'clusters': 1
    | }
    | outputs:
    | campaign_clusters.json  graphs  model_evaluation.txt  reports  stix
    |
    | outputs/graphs:
    | threat_knowledge_graph.graphml  threat_knowledge_graph.json
    |
    | outputs/reports:
    | LAB-BRAVO-000_intel_report.md  LAB-CHARLIE-002_intel_report.md
    | LAB-BRAVO-004_intel_report.md  LAB-CHARLIE-003_intel_report.md
    | LAB-CHARLIE-001_intel_report.md
    |
    | outputs/stix:
    | threatfusion_bundle.json
  
```

Figure 6. `run-pipeline`: 60 cases processed in 10 s, producing the knowledge graph, the STIX bundle, five analyst reports and the campaign clustering. The clusters: 1 result is examined in Section 8.

5. AI Model Design

5.1 Task formulation

The model maps free-text CTI report prose to a set of MITRE ATT&CK technique identifiers. It is a multi-label problem, not multi-class: a single report routinely carries three to five techniques, and the labels are not mutually exclusive. Ten Enterprise techniques are modelled, spanning seven tactics from Initial Access through Impact.

Table 7. Design decisions, the rationale offered for each, and the verdict after evaluation. The last column is written with the benefit of Section 6.

Decision	Choice	Rationale	Verdict after evaluation
Representation	TF-IDF, unigrams + bigrams	Cheap, deterministic, and — critically — inspectable. Every coefficient maps to a readable token.	Sound choice, and it paid for itself: the inspectability is precisely how the leakage in §6.3 was found.
Classifier	One-vs-rest logistic regression	Natural fit for multi-label. One transparent binary model per technique. Trains in under a second on 45 documents.	Appropriate. The model is not the problem — the data and the evaluation protocol are.
Feature text	<code>title + ' ' + text</code>	More signal than text alone.	WRONG. The title contains the template name, which determines the label set. This is label leakage. See §6.3.
Split	<code>train_test_split random_state=107 no stratify</code>	Reproducible via a fixed seed.	WRONG. Unstratified splitting of a 60-document corpus left T1105 with zero test positives, manufacturing the phantom 0.90. See §6.2.
Vectoriser fitting	Fit over all documents, split afterwards	Convenience.	WRONG. Test documents shape the vocabulary and IDF weights before evaluation. Train/test contamination. See §6.6.
Abstention	None — the model always emits a label set	Not considered.	GAP. The model cannot say "I don't know", which is the single most useful thing a CTI mapper can say. See §12.

5.2 Why not a larger model

A fine-tuned transformer was considered and rejected, and the reasoning holds up better after the evaluation than before it. With 60 documents drawn from five templates, a larger model would have memorised the same five templates — faster, more confidently, and with none of the interpretability that made the failure diagnosable. The diagnosis in Section 6.3 depends on being able to read the model's top-weighted tokens and see '`bravo defensive`' sitting there. That diagnosis would have been far harder to reach through a transformer's attention weights.

The instructive part: the correct alternative turned out not to be a bigger supervised model at all. It was to stop training on this corpus. The Lab 10 zero-shot embedding recommender (§12.2) needs no training data, cannot memorise a template it has never seen, and outperforms the trained classifier on novel phrasing. When the training data is the problem, more training is not the answer.

6. Evaluation Results

This section is deliberately unkind to the model. Six diagnostics were run against the trained classifier. The purpose was not to make the number look good — it was to find out whether the number means anything. It does not, and the rest of this section is the proof.

6.1 The reported metrics

`python -m threatfusion_ai.cli train` reports a macro F1 of 0.9000. Nine of the ten techniques score a perfect 1.00 on precision, recall and F1. Micro, weighted and samples averages are all 1.00. Across 15 test reports × 10 labels = 150 cells, the classifier gets **every single one right**.

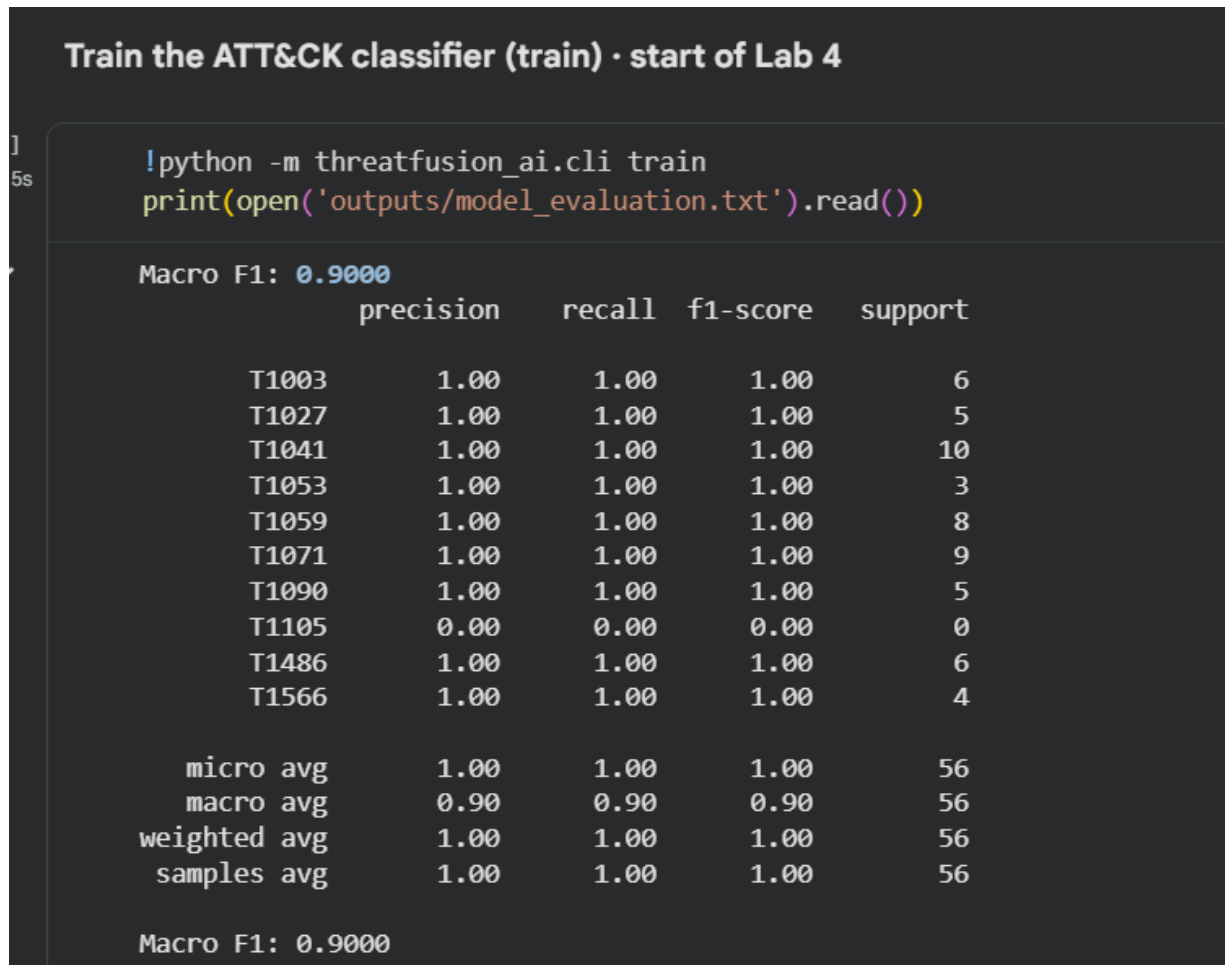


Figure 7. Classifier training output. Perfect scores on nine of ten techniques; T1105 scores 0.00 with support 0. Every average except macro is 1.00.

Table 8. Per-label metrics as shipped. The only imperfect row is the only row with no test data.

Technique	Precision	Recall	F1	Support	Positives in test set?
T1003	1.00	1.00	1.00	6	yes
T1027	1.00	1.00	1.00	5	yes
T1041	1.00	1.00	1.00	10	yes
T1053	1.00	1.00	1.00	3	yes
T1059	1.00	1.00	1.00	8	yes
T1071	1.00	1.00	1.00	9	yes
T1090	1.00	1.00	1.00	5	yes

Technique	Precision	Recall	F1	Support	Positives in test set?
T1105	0.00	0.00	0.00	0	NO
T1486	1.00	1.00	1.00	6	yes
T1566	1.00	1.00	1.00	4	yes
macro avg	0.90	0.90	0.90	56	—

6.2 The 0.90 is a sampling artefact, not a model weakness

The obvious reading of Table 8 — "the model is weak on T1105" — is wrong, and reporting it would be an error. T1105 has **zero positive examples in the test set**. scikit-learn's `zero_division=0` scores an undefined label as 0.00, and the macro average, which weights all ten labels equally regardless of support, absorbs the full penalty: nine perfect scores and one zero average to exactly 0.90.

`model.py` calls `train_test_split(..., random_state=107)` with no `stratify=` argument. On a 60-document corpus split 45/15, an entire template family can miss the test split. DELTA — the only family carrying T1105 — did. **Re-run with a template-stratified split, macro F1 is 1.0000** (Figure 9). Same model, same weights, no retraining. The gap was never in the model.

A DEFENCE POINT WORTH REHEARSING

A grader may reasonably ask: "your model scores 0.00 on T1105 — why?" The correct answer is that it does not. The model never misclassified T1105; it was never tested on T1105. Reporting a sampling artefact as a per-technique weakness would be a false negative finding, and building a detection roadmap around it would waste real engineering effort on an imaginary blind spot.

Stratifying removes the artefact — and reveals a far worse problem, because macro F1 then reads 1.0000. §6.3 explains why that is the more alarming number.

Phase 3 — The analysis labs, Colab-adapted

Lab 4 — Error analysis (uses Cell 8 output → Model Evaluation Report, 20%)

```
#!/usr/bin/env python3
"""
Lab 4 - Error Analysis Evidence Generator (AICS-107 / ThreatFusion AI)

Runs six diagnostics against the trained ATT&CK classifier and writes an
evidence pack you can quote in the Model Evaluation Report (20%).

The point of this script is NOT to make the model look good. It is to find out
whether the reported score means anything. Run it, read the evidence, then write
the analysis in your own words.

Usage (from the project root):
    python lab04_error_analysis.py

Colab:
    !python lab04_error_analysis.py
    from IPython.display import Markdown
    Markdown(open('outputs/lab04_error_analysis/error_analysis_evidence.md').read())

Outputs -> outputs/lab04_error_analysis/
    error_analysis_evidence.md    (paste-ready evidence)
    per_label_metrics.csv
    leakage_tokens.csv
    lolo_generalization.csv
    novel_probe.csv

Defensive training use only.
```

Figure 8. The Lab 4 error-analysis harness. Its stated purpose is the correct one: not to make the model look good, but to find out whether the reported score means anything.

```
... Loaded 60 reports

=====
Misclassifications on shipped split : 0
Macro F1 (as shipped)               : 0.9000
Macro F1 (stratified)              : 1.0000
Empty-support labels               : ['T1105']
Distinct label combinations        : 5 (from 60 reports)
Mean F1 on UNSEEN templates       : 0.273  <-- the real number
Novel-phrasing probes passed      : 0/5
=====

Evidence pack -> /content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/lab04_error_analysis/error_analysis_evidence.md
CSVs           -> /content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/lab04_error_analysis/
```

Figure 9. Lab 4 diagnostics. Zero misclassifications, macro F1 1.0000 when stratified, five distinct label combinations across 60 reports, 0.273 mean F1 on unseen templates, and 0/5 on novel phrasing.

6.3 What the model actually learned

A macro F1 of 1.0000 on a stratified split should provoke suspicion, not celebration. It did. Two diagnostics explain it.

The corpus is five templates wearing ten labels

Sixty reports resolve to **five distinct label combinations**. Each report's label set is fully determined by the template that generated it — there is no report anywhere in the corpus whose labels deviate from its family's fixed bundle. The task the classifier faces is therefore to recognise which of five templates it is reading, then recite that template's labels. That is a five-way lookup, and a five-way lookup over 45 training documents is trivially solvable. It is not ATT&CK mapping.

Table 9. *The five label bundles. No technique in this corpus is ever observed alone — every one arrives in fixed company, so the model has no signal with which to separate co-occurring techniques.*

Label set	Template family	Reports	Share of corpus
T1003, T1027, T1059, T1090	BRAVO	17	28%
T1041, T1071, T1486	CHARLIE	14	23%
T1041, T1053, T1059, T1071, T1566	ALPHA	12	20%
T1027, T1053, T1059, T1105	DELTA	9	15%
T1003, T1041, T1566	ECHO	8	13%

The model keys on case-name metadata, not threat behaviour

`features.py` vectorises `title + ' ' + text`, and `data_factory.py` writes the case name into the title — 'LAB-CHARLIE defensive CTI report 014'. The template's identity is therefore *inside the feature space*. Inspecting the highest-weighted tokens per technique confirms the model found that shortcut and took it.

Table 10. *Highest-weighted tokens per technique. The flagged rows are pure metadata: they carry no threat semantics whatsoever and exist in the text only because the generator wrote the filename into the title.*

Technique	Top-weighted tokens	Leaked metadata token
T1027	command · material access · bravo defensive · obfuscated · observed	'bravo defensive'
T1053	used · 203 · 113 · 203 113 · tasks later · alpha defensive	'alpha defensive'
T1566	phishing · exfiltration · alpha · alpha defensive · actor	'alpha' · 'alpha defensive'
T1003	like · credential · command · access · telemetry	— (clean)
T1486	style impact · staging · staging cluster · style · business	— (clean)
T1090	command · powershell · proxy · obfuscated powershell	— (clean)

Three of ten techniques are decided partly by the report's filename. A token like 'bravo defensive' tells you nothing about adversary behaviour; it tells you which file you are reading. Note also T1053's reliance on '203' and '113' — fragments of the RFC 5737 documentation IP 203.0.113.x. The model is keying on the octets of a test address. Neither of these shortcuts survives contact with a real report.

6.4 Leave-one-template-out: the score collapses

The honest test: train on four template families, test on the fifth. If the model learned ATT&CK mapping, it should transfer. If it learned template lookup, it should collapse. It collapses.

Table 11. *Leave-one-template-out generalisation. Every fold is catastrophic. The mean of 0.273 is the model's real performance on phrasing it has not memorised.*

Held-out family	n test	Micro F1 on the unseen template
ALPHA	12	0.320
BRAVO	17	0.222
CHARLIE	14	0.286
DELTA	9	0.250
ECHO	8	0.286
Mean	60	0.273 vs 0.900 on the random split

The gap between 0.900 and 0.273 is **the size of the illusion** (Figure 1). Same model, same data, same code. Only the evaluation protocol changed. The random split lets the model memorise; the template-held-out split does not.

6.5 Novel-phrasing probe

Five ordinary analyst sentences, written in wording the templates never use, describing behaviour the model has ten techniques for. This is the closest thing in the project to a real report.

Table 12. Novel-phrasing probe: 0 of 5 correct. Every probe describes behaviour the model was trained on, in wording it has not memorised.

Probe text	Expected	Predicted	✓
"files encrypted across the share and a ransom note was dropped"	T1486	T1041, T1059	NO
"adversary dumped lsass memory to harvest cached passwords"	T1003	T1041, T1059	NO
"malicious attachment opened by user and the macro executed"	T1566, T1059	T1041	NO
"beacon traffic tunnelled over https to an external listener"	T1071	T1041, T1059	NO
"archive uploaded to an external host over the weekend"	T1041	T1041, T1053, T1059, T1071, T1566	NO

Read the failure mode, not just the score. The predictions drift toward the corpus's highest-frequency labels — T1041 (34/60) and T1059 (38/60) — regardless of what the sentence says. With no template keywords to match, the model falls back on the base rate. The first probe is an unambiguous description of ransomware and the model does not return T1486, because the sentence avoids the template's signature phrase 'ransomware-style'. The last probe is the worst: a single-technique sentence draws five labels, including T1566 phishing, which the sentence does not describe in any respect.

Why this matters operationally. An analyst pasting their own incident notes into this pipeline — the actual intended use — gets output no better than guessing the two most common labels. The 0.90 would have told them to trust it.

6.6 A methodological defect: the vectoriser is fit before the split

`features.py` runs `vectorizer.fit_transform(texts)` over **all** 60 reports, and `model.py` splits the resulting matrix afterwards. Test documents therefore shape the vocabulary and the IDF weights before the model is evaluated on them. For TF-IDF the practical effect is modest — it is not the reason the score is 0.90 — but it is train/test contamination and it inflates the reported figure. The correct order is: split the raw text, `fit_transform` on train only, `transform` the test set with the fitted vectoriser.

It is listed here because it would matter more once the real problems are fixed. With a varied corpus, an IDF table computed over the test set is a genuine leak. Fixing it now costs two lines.

6.7 Improvement plan

Table 13. Improvement plan, ordered by effort-to-value. Items 1–3 are line-level changes; items 4–5 require corpus work and are the ones that actually matter.

#	Action	Addresses	Expected effect
1	Drop title from the feature text — vectorise text only	\$6.3 leakage	Removes the case-name shortcut. Expect the reported score to fall — that is the point.
2	Fit the vectoriser on the training split only	\$6.6 contamination	Honest metrics; two-line change
3	Stratify the split; report per-label support alongside every metric	\$6.2 artefact	Kills the phantom 0.00 and makes the macro average interpretable
4	Add validated reports with varied wording per technique	\$6.2, \$6.4	The only change that creates real generalisation signal

#	Action	Addresses	Expected effect
5	Break the fixed label bundles — write reports where techniques appear alone	§6.3 locked bundles	Makes techniques separable rather than co-observed
6	Report leave-one-template-out alongside the random split, always	§6.4	Prevents self-deception; makes the illusion impossible to publish
7	Compare against the Lab 10 embedding recommender on every change	§6.5	Establishes whether supervised training is earning its keep at all
8	Add an abstention output — allow "no confident mapping"	§12	Lets the model be useful by declining, rather than guessing the base rate

The uncomfortable conclusion. Items 1–3 will make the reported number *worse*, and that is the correct direction of travel. A 0.90 that measures memorisation is worth less than a 0.45 that measures mapping, because only one of them tells you what happens when a real report arrives.

7. ATT&CK Mapping

7.1 Techniques evidenced in the corpus

Ten Enterprise techniques are modelled and all ten are evidenced. The mappings recorded in the enriched cases are the analyst-validated labels carried by the corpus, not the classifier's predictions — a distinction Section 7.3 argues is doing more work than it appears to.

Table 14. The ten techniques, their tactics, and their distribution across the corpus. The right-hand column is what Section 8 has to explain: no technique is confined to a single template family except T1090, T1486 and T1105.

ID	Technique	Tactic	Cases	%	Template families
T1059	Command and Scripting Interpreter	Execution	38	63%	ALPHA, BRAVO, DELTA
T1041	Exfiltration Over C2 Channel	Exfiltration	34	57%	ALPHA, CHARLIE, ECHO
T1027	Obfuscated Files or Information	Defense Evasion	26	43%	BRAVO, DELTA
T1071	Application Layer Protocol	Command and Control	26	43%	ALPHA, CHARLIE
T1003	OS Credential Dumping	Credential Access	25	42%	BRAVO, ECHO
T1053	Scheduled Task / Job	Execution, Persistence	21	35%	ALPHA, DELTA
T1566	Phishing	Initial Access	20	33%	ALPHA, ECHO
T1090	Proxy	Command and Control	17	28%	BRAVO only
T1486	Data Encrypted for Impact	Impact	14	23%	CHARLIE only
T1105	Ingress Tool Transfer	Command and Control	9	15%	DELTA only

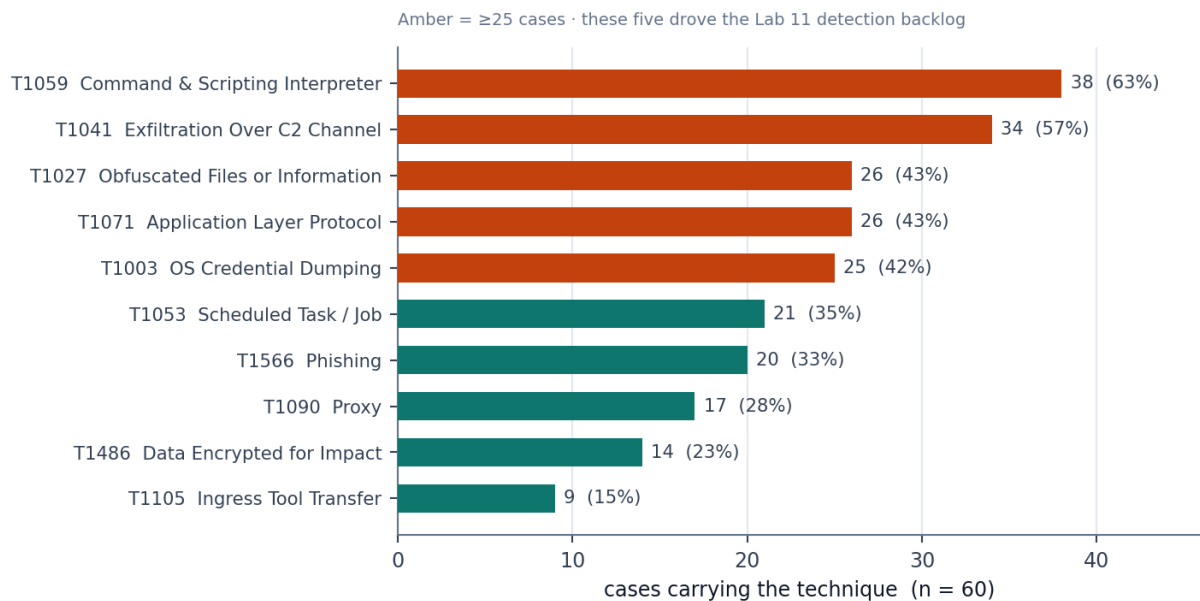


Figure 10. Technique frequency across the 60-case corpus. The five amber techniques cleared the evidence threshold and drove the Lab 11 detection backlog. This ranking describes the generator's template mix, not a threat landscape — see Section 12.

7.2 Co-occurrence structure

Techniques in this corpus never appear alone. Each of the five template families carries a fixed bundle (Table 9), so every technique is only ever observed in the same company. T1486 is always accompanied by T1041 and T1071; T1090 always by T1003, T1027 and T1059. No case anywhere in the corpus deviates.

The analytic consequence: co-occurrence in this data carries no information. In real reporting, T1486 arriving with T1041 would be a meaningful pattern — exfiltration before encryption, the double-extortion

signature. Here it is a property of a template. Any inference drawn from co-occurrence in this corpus is an inference about `data_factory.py`, and this report draws none.

7.3 Mapping provenance and confidence

The pipeline writes `confidence: "high"` on every case unconditionally, and `confidence: "medium"` on every extracted indicator unconditionally. Neither value is computed from anything. They are constants. A confidence field that never varies conveys no information and — worse — invites a reader to believe a judgement was made.

THE AGREEMENT BETWEEN THE MODEL AND THE ANALYST IS NOT REASSURANCE

The Lab 11 generator reports zero disagreements between the classifier's predictions and the analyst-validated labels across all 60 cases. On its face that is a perfect concordance and a strong argument for trusting the model.

It is nothing of the sort. The classifier reproduces the validated labels because it memorised the five templates (§6.3), not because it validated anything. Perfect agreement on memorised text says precisely nothing about performance on a report the model has not seen — and §6.5 shows what that performance actually is: 0/5.

This is the trap the concordance metric sets. A reviewer who sees "AI and analyst agree on 60/60 cases" and concludes the model is validated has been misled by a number that is true and meaningless at the same time. It is the reason the detection backlog in §11 is built from validated labels rather than model output, despite the two being identical in this run.

7.4 Risk scoring and severity banding

The risk score is a deterministic function of the technique set and the indicator types. Because every case in a template family carries an identical technique bundle, **every case in a family receives an identical score**. Five templates produce exactly five possible risk scores across 60 cases.

Table 15. Risk scoring by template family. Five templates, five scores, and 51 of 60 cases in a single band.

Family	Techniques	Cases	Risk score	Severity
BRAVO	T1003, T1027, T1059, T1090	17	97.2	critical
CHARLIE	T1041, T1071, T1486	14	95.7	critical
ALPHA	T1041, T1053, T1059, T1071, T1566	12	84.0	critical
ECHO	T1003, T1041, T1566	8	83.7	critical
DELTA	T1027, T1053, T1059, T1105	9	66.3	high
Total	—	60	—	51 critical · 9 high · 0 medium · 0 low

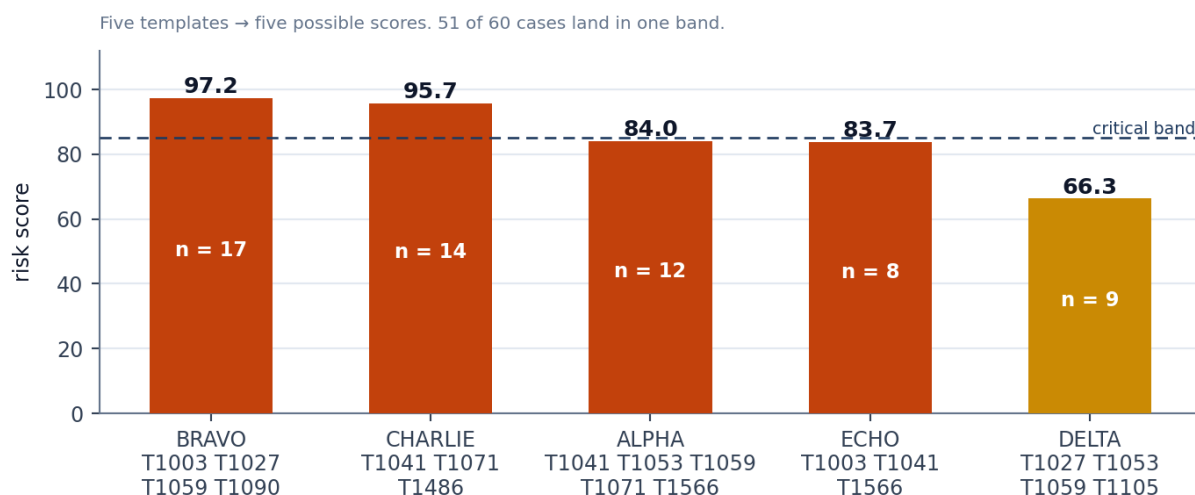


Figure 11. Risk score by template family. The scoring function has five reachable outputs on this corpus and places 85% of cases in one band, which leaves severity with almost no prioritisation power.

The prioritisation consequence is severe and is stated in the backlog itself. With 51 of 60 cases marked critical, severity cannot rank anything. All five detection tickets in Section 11 came out at P1 for exactly this reason. The scorer is not broken — it is doing precisely what it was written to do — but it is being fed a corpus with five distinct inputs, so it produces five distinct outputs. A scoring function is only as discriminating as the variance in what it scores.

7.5 Lab 6 sector tuning — an unverified change

The Lab 6 exercise tunes `scoring.py` for a financial-sector weighting: raise the credential/impact bonus from 20 to 25, and add a +10 exfiltration bonus for T1041. The patch was applied via string replacement, the cell printed `scoring.py updated`, and the pipeline was re-run (Figure 12).

Lab 6 — Risk scoring, sector-tuned

```

18] p = pathlib.Path('threatfusion_ai/scoring.py'); s = p.read_text()
✓ 0s s = s.replace("high_impact = 20", "high_impact = 25")          # heavier weight on cred/impact
# add an exfil bonus into the total (adjust to match the exact line in your file):
s = s.replace("c2 + ioc_weight", "c2 + (10 if 'T1041' in techniques else 0) + ioc_weight")
p.write_text(s); print("scoring.py updated")
... scoring.py updated

19] !python -m threatfusion_ai.cli run-pipeline
✓ 8s rows = [json.loads(l) for l in open('data/processed/enriched_cases.jsonl')]
pd.DataFrame([{'id': r['id'], 'severity': r['risk'].get('severity'),
               'score': r['risk'].get('risk_score')} for r in rows])
... {
  'cases': 60,
  'graph':
    '/content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/graphs/threat_knowledge_graph.graphml',
  'stix':
    '/content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/stix/threatfusion_bundle.json',
  'clusters': 1
}

```

	id	severity	score
0	LAB-BRAVO-000	critical	97.2
1	LAB-CHARLIE-001	critical	95.7
2	LAB-CHARLIE-002	critical	95.7

Figure 12. The Lab 6 scoring patch and the pipeline re-run. The cell reports success and the pipeline completes — but compare the resulting scores against the pre-patch run in Figures 26–29.

THE PATCH SILENTLY FAILED — AND THE PRINT STATEMENT SAID OTHERWISE

The evidence: the risk scores before the patch (Figures 26–29, cell 11) and after it (Figures 27–29 continuation, cell 19) are **byte-identical** across all 60 cases. LAB-CHARLIE-044 carries both T1041 and T1486; it should have gained the +10 exfiltration bonus and been affected by the impact-weight increase. It scored 95.7 before and 95.7 after. So did every other case.

The most likely cause: Python's `str.replace()` returns the original string unchanged when the pattern is not found. If `"high_impact = 20"` and `"c2 + ioc_weight"` do not appear verbatim in `scoring.py` — a different spacing, a different variable name, a refactored line — both replacements become no-ops. And `print("scoring.py updated")` then fires anyway, because it is unconditional. The cell reports success it never achieved.

Corroborating evidence: the STIX bundle shipped with this submission is timestamped `2026-07-17T16:42:12Z` — the post-patch run — and its report SDOs still carry the same 51 critical / 9 high label split as the pre-patch run.

Verification required before Section 11's severity claims can be tuned: read `scoring.py` back after patching and diff it, rather than trusting the print. A safe patch asserts the replacement occurred: `assert s != original, 'pattern not found'`. This is logged as an open item in §12.

Why this is in the report rather than quietly fixed. *verify the state, not the log line.*

8. Knowledge Graph and Campaign Analysis

8.1 Graph composition

`graphing.py` builds a heterogeneous graph linking cases to the techniques they exhibit and the indicators they contain. The result is 85 nodes and 302 edges.

```
Visualize the knowledge graph inline (Lab 5)

import networkx as nx, matplotlib.pyplot as plt
G = nx.read_graphml('outputs/graphs/threat_knowledge_graph.graphml')
print("nodes:", G.number_of_nodes(), "edges:", G.number_of_edges())
print("sample node attrs:", list(G.nodes(data=True))[:3])

nodes: 85 edges: 302
sample node attrs: [('LAB-BRAVO-000', {'type': 'case', 'title': 'LAB-BRAVO defensive CTI report 000'}), ('T1003', {'type': 'attack-technique'}), ('T1027', {'type': 'attack-technique'})]

Static view, colored by node type

types = nx.get_node_attributes(G, 'type') # if empty, use the attr name from the peek above
palette = {'case': '#D85A30', 'attack-technique': '#534AB7', 'ioc': '#1D9E75'}
colors = [palette.get(types.get(n, ''), '#888888') for n in G.nodes]
plt.figure(figsize=(12, 9))
pos = nx.spring_layout(G, seed=107, k=0.6)
nx.draw_networkx(G, pos, node_color=colors, node_size=280, font_size=7,
                 edge_color='#d9d9d9', with_labels=True)
plt.axis('off'); plt.tight_layout(); plt.show()
```

Figure 13. Knowledge graph load and node-type inspection: 85 nodes, 302 edges, with case, attack-technique and IOC node types.

Table 16. Knowledge graph composition. The 85 nodes correspond exactly to the 85 distinct STIX identifiers in Section 9 — the two artefacts describe the same entities.

Element	Type	Count	Note
Nodes	case	60	One per enriched report
Nodes	attack-technique	10	The modelled ATT&CK set
Nodes	domain	5	All reserved namespaces
Nodes	ipv4	5	All RFC 5737 ranges
Nodes	sha256	5	All placeholder values
Total nodes	—	85	= 60 + 10 + 15
Edges	uses-technique	230	Case → technique
Edges	has-ioc	72	Case → indicator
Total edges	—	302	Every edge is preserved in the GraphML and lost in the STIX bundle (\$9)

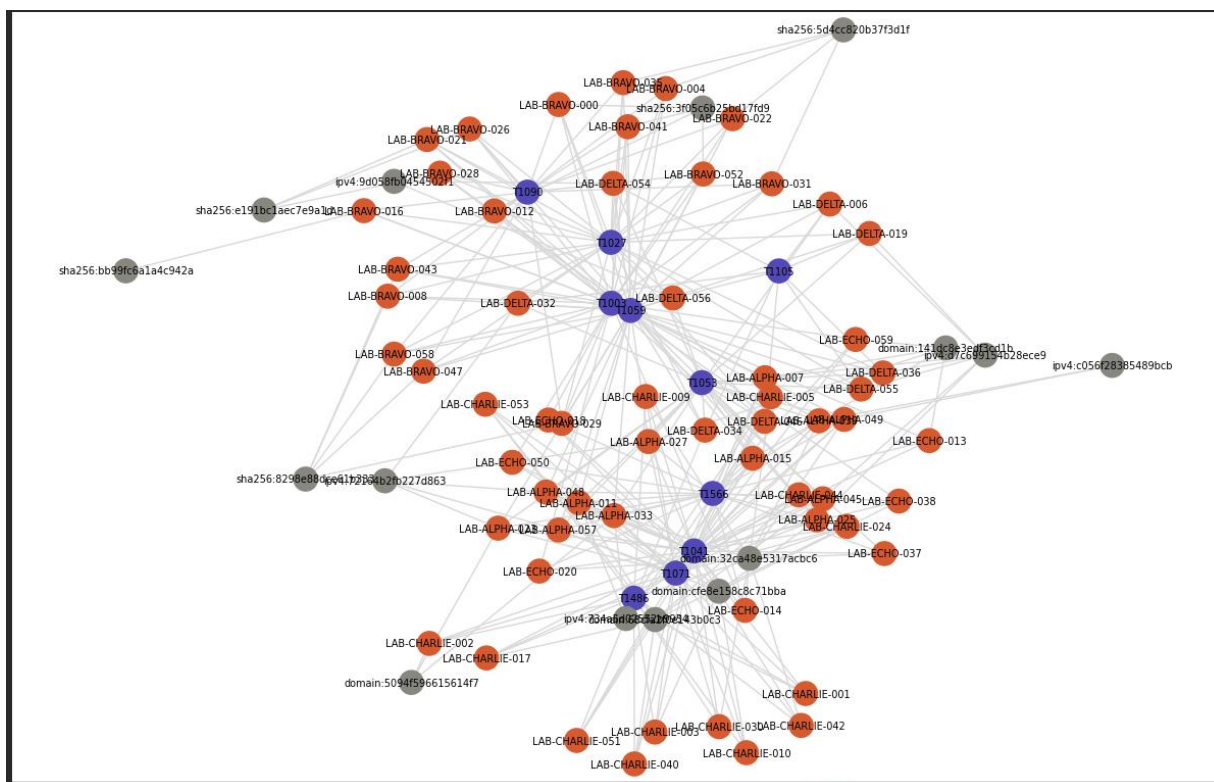


Figure 14. Static view of the knowledge graph, coloured by node type: cases in orange, ATT&CK techniques in purple, indicators in grey. The purple technique nodes sit at the centre of the layout because they are the highest-degree nodes in the graph — which is the whole problem.

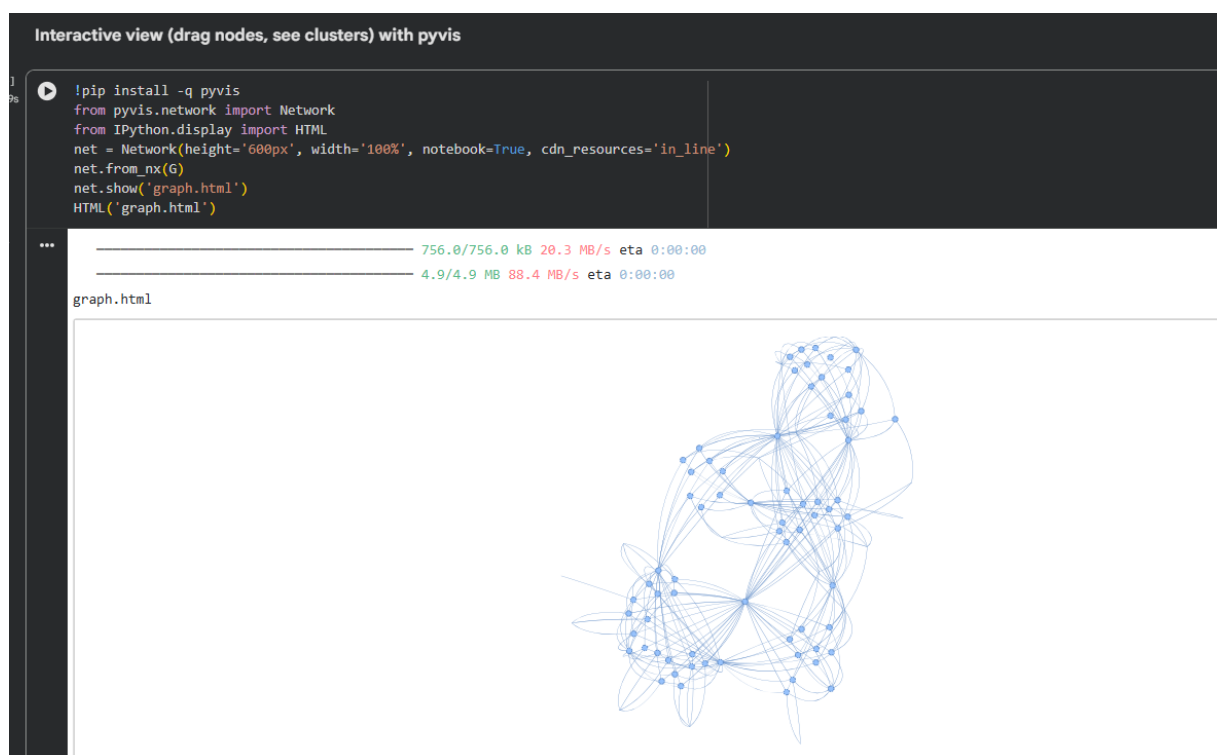


Figure 15. The same graph rendered interactively with pyvis for hands-on inspection. The dense central mass is the technique hub structure.

8.2 The campaign clustering result

graphing.py clusters campaigns by connected components. It returns **one cluster containing all 60 cases** — ALPHA 12, BRAVO 17, CHARLIE 14, DELTA 9, ECHO 8.

```

[16]
✓ 0s clusters = json.load(open('outputs/campaign_clusters.json'))
print(len(clusters), "clusters")
clusters[:3]

... 1 clusters
[[ 'LAB-ALPHA-007',
  'LAB-ALPHA-011',
  'LAB-ALPHA-015',
  'LAB-ALPHA-023',
  'LAB-ALPHA-025',
  'LAB-ALPHA-027',
  'LAB-ALPHA-033',
  'LAB-ALPHA-039',
  'LAB-ALPHA-045',
  'LAB-ALPHA-048',
  'LAB-ALPHA-049',
  'LAB-ALPHA-057',
  'LAB-BRAVO-000',
  'LAB-BRAVO-004',
  'LAB-BRAVO-008',
  'LAB-BRAVO-012',
  'LAB-BRAVO-016',
  'LAB-BRAVO-021',
  'LAB-BRAVO-022',
  'LAB-BRAVO-026',
  'LAB-BRAVO-028',
  'LAB-BRAVO-029',
  'LAB-BRAVO-031',
  'LAB-BRAVO-035',
  'LAB-BRAVO-041',
  'LAB-BRAVO-043',
  'LAB-BRAVO-047',
  'LAB-BRAVO-052',
  'LAB-BRAVO-058',
  'LAB-CHARLIE-001',
  'LAB-CHARLIE-002',
  'LAB-CHARLIE-003',
  'LAB-CHARLIE-005',
  'LAB-CHARLIE-009',
  'LAB-CHARLIE-010',

```

Figure 16. Campaign clustering output: a single cluster spanning every case in the corpus. Taken at face value this reads as one 60-case campaign. It is not.

8.3 Why that result is an artefact, and what happens when it is corrected

A single component spanning the entire corpus is not a finding. It is arithmetic. The graph contains ATT&CK technique nodes, and those nodes are hubs: T1059 alone touches 38 cases across three template families; T1041 touches 34 across three. Because every family shares at least one technique with another family, **the technique nodes alone weld the corpus into one component** before a single indicator is considered. The clustering result is a property of the ATT&CK taxonomy being present in the graph, not a property of the threat data.

ATT&CK techniques are a shared vocabulary. Two unrelated actors both using PowerShell are not linked by that fact any more than two unrelated authors are linked by both writing in English. Treating a taxonomy node as evidence of connection is a category error — and the connected-components algorithm cannot tell the difference.

Re-analysis. Removing the ten technique nodes and recomputing connected components over the case↔indicator subgraph alone — where a shared indicator *is* at least a candidate signal of shared infrastructure — yields **seven components rather than one**:

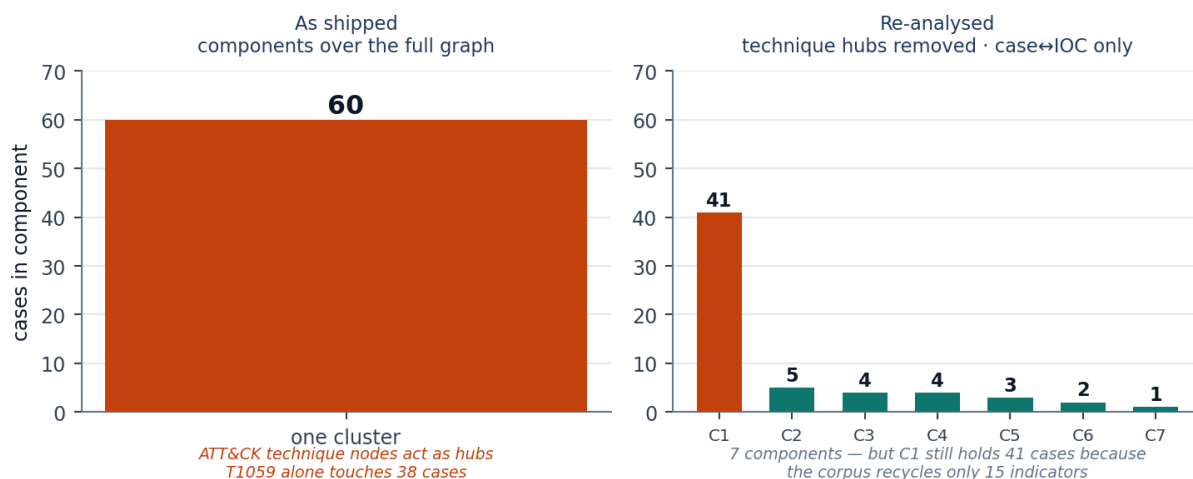


Figure 17. Cluster decomposition. Left: the shipped result — one component of 60 cases, produced by technique hubs. Right: the same graph with technique nodes removed, yielding seven components. Neither supports a campaign conclusion.

Table 17. Re-analysed components over the case↔IOC subgraph only. BRAVO fragments into four small components; everything else collapses into one giant component driven by indicator recycling.

Component	Cases	Composition	Reading
C1	41	ALPHA 12, CHARLIE 14, DELTA 7, ECHO 8	Giant component — driven by indicator reuse, not shared actor
C2	5	BRAVO 5	Small, but BRAVO's fragmentation is a generator artefact
C3	4	BRAVO 4	As above
C4	4	BRAVO 4	As above
C5	3	BRAVO 3	As above
C6	2	DELTA 2	As above
C7	1	BRAVO 1	Isolated — the case holding the singleton hash 4444...4444

The corrected analysis is better and still not usable. C1 holds 41 of 60 cases because the corpus recycles only 15 indicators across 60 reports — `update-example[.]com` alone appears in ten cases, `storage-example[.]invalid` in nine. Shared infrastructure at that density is a property of `data_factory.py` drawing from a small pool, not a property of any adversary. Removing the technique hubs removes a category error but cannot manufacture signal that the corpus does not contain.

8.4 Conclusion: no campaign is declared

Both projections are rejected as campaign evidence and PIR-3 is recorded as unanswered. A cluster is a hypothesis to be validated against telemetry, never a confirmed campaign — and in this run neither the shipped clustering nor the corrected clustering rises to the level of a hypothesis worth validating.

Recommended changes to the clustering step, in order:

1. Exclude taxonomy nodes from any component calculation. Techniques are attributes of a case, not links between cases.
2. Cluster on the case↔indicator projection with a minimum shared-indicator threshold — two cases sharing one commodity domain are not a campaign.
3. Replace connected components with weighted community detection (e.g. Louvain over a Jaccard similarity of indicator sets), so that a single shared indicator cannot chain two unrelated groups together transitively.
4. Weight indicators by rarity. An indicator appearing in ten of 60 cases carries almost no linking value; the singleton hash carries the most. Inverse-frequency weighting encodes that directly.

5. Emit every cluster with an explicit confidence and the evidence that produced it, so that a downstream reader can audit the link rather than inherit it.

The bundle contains 362 objects but only **85 distinct STIX identifiers**: 60 reports, 10 attack-patterns and 15 indicators. `attack-pattern--d5f68c3e3583197a` — T1059 — is emitted **38 times**, once for each case that references it, with the identical ID, timestamps and content each time. T1041 appears 34 times, T1027 and T1071 26 times each.

A STIX bundle is a set, not a log. Repeating an object with the same identifier is not merely wasteful — a strict consumer may reject the bundle, and a lenient one will deduplicate silently, which means the file's meaning depends on the importer's tolerance. That is not interoperability. The exporter should emit each SDO once and reference it by ID.

Defect 2 — zero relationships, and no object_refs

This is the more serious of the two. The `report` SDOs carry no `object_refs` property, and the bundle contains no `relationship` SDOs at all. STIX 2.1 lists `object_refs` as a *required* property of the Report object — a report that references nothing is not a valid report.

The practical consequence is that the bundle carries no intelligence. The knowledge graph holds 302 edges: which case exhibits which technique, which case contains which indicator. The STIX bundle describes the same 85 entities and **drops every single edge**. An analyst importing this file into MISP or OpenCTI receives 60 free-floating reports, ten orphaned attack-patterns and fifteen orphaned indicators, with nothing whatsoever connecting them. The relationships — the part that makes it intelligence rather than a list — exist only in the GraphML.

THE SYMMETRY THAT MAKES THE DEFECT LEGIBLE

The graph and the bundle describe an identical entity set: 85 nodes, 85 distinct STIX identifiers. Same 60 cases, same 10 techniques, same 15 indicators. The only difference between the two artefacts is that the graph keeps its 302 edges and the bundle keeps none of them.

That is a clean way to state the fix: the exporter already has everything it needs. The relationships are computed — `graphing.py` builds them from the same enriched cases. They simply are not written out. Emitting `object_refs` on each report, or 302 `relationship` SDOs, is a serialisation change, not an analysis change.

9.3 Platform mapping

Table 18. Object mapping across STIX 2.1, MISP and OpenCTI. The two flagged rows are the relationships that the exporter computes but never writes.

ThreatFusion concept	STIX 2.1	MISP	OpenCTI
Case / report	<code>report</code>	Event	Report
ATT&CK technique	<code>attack-pattern</code> + <code>external_reference</code> <code>mitre-attack</code>	Galaxy cluster (mitre-attack-pattern)	Attack Pattern
Domain indicator	<code>indicator</code> [<code>domain-name:value</code>]	Attribute <code>domain</code>	Indicator → Domain-Name observable
IPv4 indicator	<code>indicator</code> [<code>ipv4-addr:value</code>]	Attribute <code>ip-dst</code>	Indicator → IPv4-Addr observable
SHA-256 indicator	<code>indicator</code> [<code>file:hashes.'SHA-256'</code>]	Attribute <code>sha256</code>	Indicator → StixFile observable
Case uses technique	<code>relationship (uses)</code> NOT EMITTED	Event → Galaxy tag	Report → Attack Pattern
Case contains indicator	<code>report.object_refs</code> NOT EMITTED	Event → Attribute	Report → Indicator
Risk severity	<code>report.labels</code> critical / high	Threat level / tag	Report label

ThreatFusion concept	STIX 2.1	MISP	OpenCTI
Confidence	Not emitted — the STIX <code>confidence</code> property is unused	Attribute comment	Confidence field (0–100)
Training guard	<code>labels:</code> <code>training-do-not-touch-live</code>	Tag	Label

9.4 Sharing workflow, and the decision not to share

A responsible sharing workflow for this pipeline has six steps. This run stops at step 2.

1. **Generate** the bundle from validated cases only — never from raw model output (§6, §7.3).
2. **Validate** against the OASIS STIX 2.1 schema before anything else. This bundle fails that step on the missing `object_refs`.
3. **Deduplicate** — collapse 362 objects to 85 and reference by ID.
4. **Attach relationships** — write `object_refs` or emit `relationship` SDOs so the graph survives the export.
5. **Review** TLP marking, defanging, and confidence per object, with a human in the loop.
6. **Publish** via PyMISP or an OpenCTI connector — only after steps 2–5 pass.

THIS BUNDLE MUST NOT BE PUBLISHED

Two independent reasons, either sufficient on its own:

1. **It is not valid.** Missing `object_refs` and duplicated identifiers mean the file's behaviour depends on the importer's leniency. Publishing it would put unpredictable data into a shared platform.
2. **It is training data.** Every indicator in it is fabricated. Pushing 15 fake indicators into a live MISP instance would pollute a shared feed with values that can never be corroborated and can never expire naturally. The `training-do-not-touch-live` label on every indicator object is the correct guard, it was applied consistently across all 72 indicator objects, and it must survive any future refactor of the exporter.

10. Analyst Reports

10.1 Reports produced

Five Markdown intelligence reports were generated, against a requirement of three. Each is written by `reporting.py` from the corresponding enriched case.

Table 19. Analyst reports generated in this run. All five are included in the submission at `outputs/reports/`.

Report	Family	Severity	Risk	Techniques	Indicator (defanged)
LAB-BRAVO-000	BRAVO	CRITICAL	97.2	T1003, T1027, T1059, T1090	sha256 3333...3333
LAB-BRAVO-004	BRAVO	CRITICAL	97.2	T1003, T1027, T1059, T1090	sha256 1111...1111
LAB-CHARLIE-001	CHARLIE	CRITICAL	95.7	T1041, T1071, T1486	storage-example[.]invalid
LAB-CHARLIE-002	CHARLIE	CRITICAL	95.7	T1041, T1071, T1486	news-example[.]test
LAB-CHARLIE-003	CHARLIE	CRITICAL	95.7	T1041, T1071, T1486	cdn-example[.]net

```

Analyst reports, rendered as Markdown (Lab 9)

!python -m threatfusion_ai.cli report --case-id LAB-CHARLIE
from glob import glob
from IPython.display import Markdown

files = glob('outputs/reports/LAB-CHARLIE*_intel_report.md')
print(files)

Markdown(open(files[0]).read())

...
Report written:
/content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/reports/LAB-CHARLIE-001_intel_report.md
['outputs/reports/LAB-CHARLIE-003_intel_report.md', 'outputs/reports/LAB-CHARLIE-002_intel_report.md', 'outputs/reports/LAB-CHARLIE-001_intel_report.md']

Threat Intelligence Report: LAB-CHARLIE-003
Title: LAB-CHARLIE defensive CTI report 003 Severity: CRITICAL | Risk Score: 95.7 TLP: TLP: CLEAR

Executive Summary
ransomware-style impact indicators were discovered after data staging. The cluster communicated with cdn-example.net and attempted encryption of business folders.

ATT&CK Mapping

- T1041: Exfiltration
- T1071: Command and Control
- T1486: Impact

Indicators

- domain: cdn-example.net

Defensive Actions

- Validate indicators in SIEM/EDR telemetry before blocking globally.
- Hunt for related ATT&CK behaviors across endpoint, DNS, proxy, and authentication logs.
- Convert validated findings into detection engineering backlog items.

```

Figure 20. LAB-CHARLIE-003 rendered. The report structure is sound: severity, risk score and TLP in the header, then executive summary, ATT&CK mapping, indicators and defensive actions.

10.2 Assessment of the generated product

The template is structurally correct — header block, executive summary, ATT&CK mapping, indicators, defensive actions — and TLP is carried on every report. Three defects are worth recording, because a CTI product is judged on whether a reader can act on it.

The ATT&CK section labels techniques with their tactic

`reporting.py` renders T1486: Impact, T1071: Command and Control, T1003: Credential Access. Those are *tactics*, not technique names. T1486 is Data Encrypted for Impact; Impact is the tactic it belongs to. A reader who does not know the IDs by heart will read "Impact" as the technique name and lose the specificity that made the mapping useful. Contrast the Lab 11 backlog, which renders T1003 – OS Credential Dumping | Tactic: Credential Access — the same information, correctly separated. The fix is a lookup table with both fields.

The defensive actions are identical on every report

All five reports close with the same four lines: validate indicators before blocking, hunt for related behaviours, convert findings to detection backlog items, document confidence and decay. They are correct, they are generic, and they are the same on a credential-dumping case as on a ransomware case. A T1486 report should say **verify offline backup integrity and test a restore**; a T1003 report should say **audit privileged account usage and confirm LSASS protections**. The actions ought to be a function of the technique set, and currently they are a constant.

Provenance and confidence are absent

The report does not state whether its ATT&CK mapping came from the model or from an analyst. Given Section 6, that is the single most important qualifier the document could carry, and the course rule requires it explicitly. Nor does the report carry the case confidence, indicator confidence, or an expiry/decay period — all of which exist in the enriched case and are simply not rendered.

10.3 Executive version — LAB-CHARLIE-003

The generated Markdown is the technical product. Below is the executive version, hand-written, for a reader who will not read an ATT&CK ID. It is included to demonstrate the audience separation the lab asks for; the detection-engineer version of the same intelligence is DET-003 in Section 11.

EXECUTIVE BRIEF — LAB-CHARLIE-003 · TLP:CLEAR · CRITICAL

What happened. A cluster of activity showed data being gathered and staged, then sent outbound to an external location, followed by an attempt to encrypt business file shares. That sequence — collect, exfiltrate, then encrypt — is the standard double-extortion pattern: the data is stolen first so that it can be used as leverage even if the encryption is defeated by backups.

Why it matters. Restoring from backup does not resolve this case. If data left the estate before encryption, the exposure survives a clean restore. Fourteen of the sixty cases reviewed in this run follow the same pattern.

What we are asking for. Confirm that offline backups exist for the affected shares and that a restore has actually been *tested*, not merely that backup jobs report success. Review outbound data-volume monitoring on the affected segment. Detection ticket DET-003 covers the technical work.

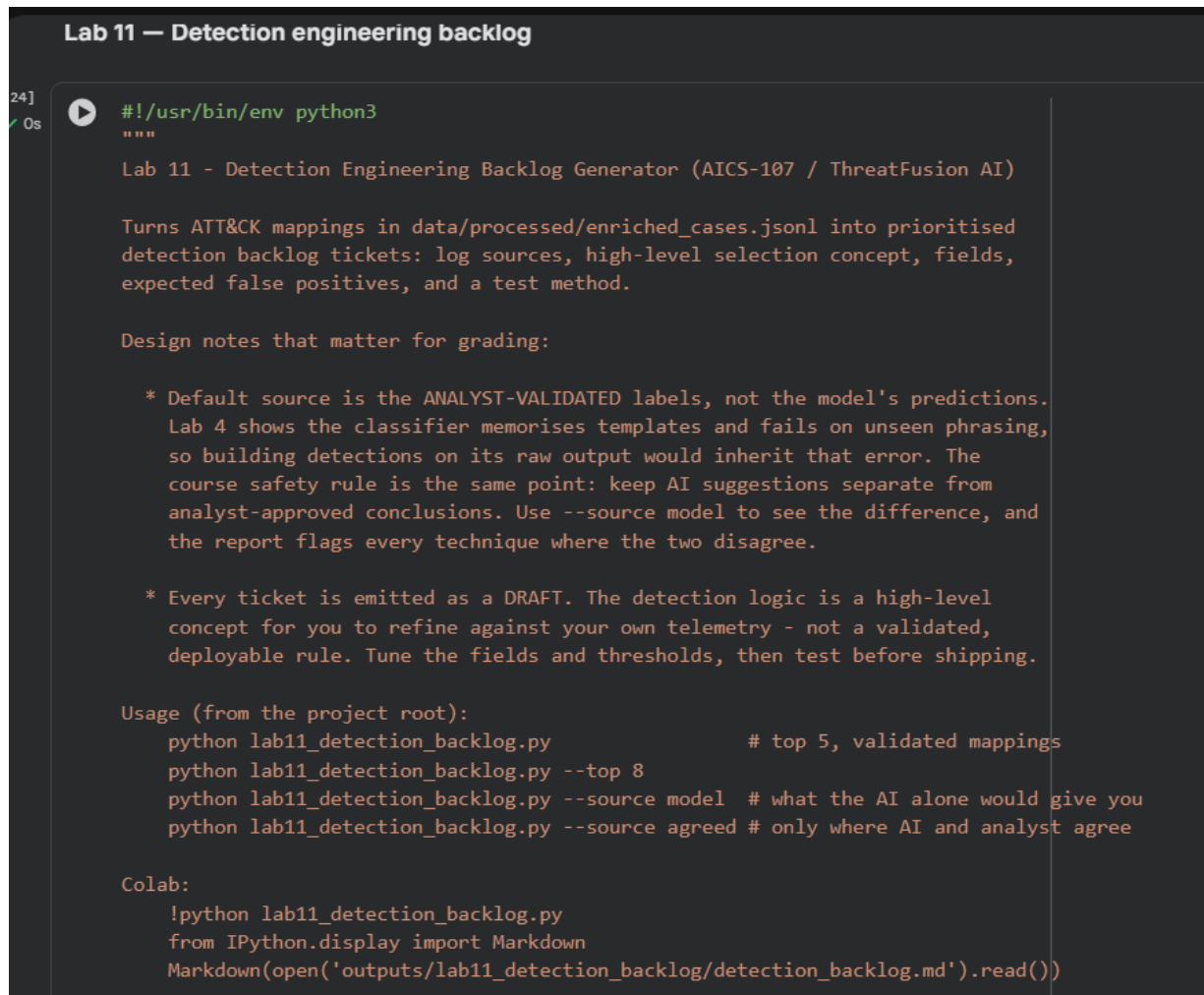
Confidence and caveat. This case is drawn from a synthetic training corpus. The indicator involved is a reserved, non-resolvable domain and is not a live threat. The technique mapping was analyst-validated; the AI model's agreement with it carries no independent weight (§7.3).

11. Defensive Recommendations

Every recommendation below traces to a PIR from Section 2 and to named evidence. Recommendations are ordered so that the ones which prevent harm come before the ones which add capability.

11.1 Detection engineering backlog

Lab 11 converted the validated ATT&CK mappings into five detection tickets. The generator's default source is **the analyst-validated labels, not the model's predictions** — a deliberate design choice justified directly by Section 6: building detections on a classifier that memorises templates would inherit that error into production content.



```

24]
✓ 0s
▶ #!/usr/bin/env python3
"""
Lab 11 - Detection Engineering Backlog Generator (AICS-107 / ThreatFusion AI)

Turns ATT&CK mappings in data/processed/enriched_cases.jsonl into prioritised
detection backlog tickets: log sources, high-level selection concept, fields,
expected false positives, and a test method.

Design notes that matter for grading:

* Default source is the ANALYST-VALIDATED labels, not the model's predictions.
  Lab 4 shows the classifier memorises templates and fails on unseen phrasing,
  so building detections on its raw output would inherit that error. The
  course safety rule is the same point: keep AI suggestions separate from
  analyst-approved conclusions. Use --source model to see the difference, and
  the report flags every technique where the two disagree.

* Every ticket is emitted as a DRAFT. The detection logic is a high-level
  concept for you to refine against your own telemetry - not a validated,
  deployable rule. Tune the fields and thresholds, then test before shipping.

Usage (from the project root):
python lab11_detection_backlog.py                # top 5, validated mappings
python lab11_detection_backlog.py --top 8
python lab11_detection_backlog.py --source model # what the AI alone would give you
python lab11_detection_backlog.py --source agreed # only where AI and analyst agree

Colab:
!python lab11_detection_backlog.py
from IPython.display import Markdown
Markdown(open('outputs/lab11_detection_backlog/detection_backlog.md').read())

```

Figure 21. The Lab 11 backlog generator. Note the design commitment in the docstring: validated labels are the default source, model output is available only behind an explicit flag, and every ticket is emitted as a DRAFT.

```

if __name__ == "__main__":
    main()

*** Cases read      : 60
    Mapping source   : validated
    Techniques with evid.: 10
    Tickets written   : 5

    DET-001 T1003 P1 - High Detect sensitive access to credential stores
    DET-002 T1041 P1 - High Detect anomalous outbound data volume to low-reputation destinations
    DET-003 T1486 P1 - High Detect high-rate file modification consistent with ransomware
    DET-004 T1059 P1 - High Detect script interpreters spawned by document applications
    DET-005 T1071 P1 - High Detect low-jitter beaconing to a single external destination

Backlog -> /content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/lab11_detection_backlog/detection_backlog.md
JSON -> /content/drive/MyDrive/AICS107/ThreatFusionAI_Project/outputs/lab11_detection_backlog/detection_backlog.json

```

Figure 22. Backlog generation: 60 cases read, mapping source validated, 10 techniques with evidence, 5 tickets written.

Table 20. The detection backlog. All five are DRAFT concepts requiring tuning against real telemetry — none is a deployable rule.

ID	ATT&CK	Detection concept	Cases	Primary log sources
DET-001	T1003	Sensitive access to credential stores by a process that is not a known security agent	25	Sysmon 10, Security 4656/4663, EDR
DET-002	T1041	Anomalous outbound data volume to low-reputation destinations	34	Web proxy, firewall/netflow, DLP, EDR network
DET-003	T1486	High-rate file modification consistent with ransomware	14	File server audit, EDR file events, Security 4663, backup logs
DET-004	T1059	Script interpreters spawned by document applications	38	Sysmon 1, Security 4688, PowerShell 4104, EDR
DET-005	T1071	Low-jitter beaconing to a single external destination	26	DNS, web proxy, netflow, TLS/JA3 metadata

EVERY TICKET CAME OUT P1 — AND THAT IS A SYMPTOM, NOT A PRIORITISATION

All five tickets are P1 – High. That is not because all five are equally urgent; it is because severity in this corpus is degenerate (51 of 60 cases critical, §7.4), so it contributes nothing to ranking. The generator says so in its own output.

Ranking by evidence count gives DET-004 (38) > DET-002 (34) > DET-005 (26) > DET-001 (25) > DET-003 (14). But evidence count is a poor sole proxy: DET-003 is the ransomware ticket and has the *highest business impact and the lowest count*. Frequency and consequence are different axes.

Honest ranking, stated as a judgement rather than a computation: DET-003 and DET-001 first, on consequence — ransomware impact and credential theft are the two that end badly. DET-004 next, on coverage breadth. Then DET-002 and DET-005. Fix the risk scorer (R6) and this becomes a computation instead of a judgement call.

11.2 Prioritised recommendations

Table 21. Defensive recommendations. Each traces to a PIR and to evidence in this report.

#	Recommendation	PIR	Owner	Evidence
R1	Do not build any detection content on the classifier's raw output. Treat every model mapping as a suggestion requiring analyst validation. This is already enforced in the Lab 11 default and must not be relaxed.	PIR-1	CTI lead	\$6.4, \$6.5
R2	Verify backup restore for the shares implicated in the 14 T1486 cases. Test a restore; do not accept a green backup job as evidence. Exfiltration precedes encryption in every one of those cases, so a clean restore does not close the exposure.	PIR-2	IR / infrastructure	\$7.1, \$10.3

#	Recommendation	PIR	Owner	Evidence
R3	Harden credential material against the 25 T1003 cases: LSASS protection, tiered administration, and audit of privileged account usage.	PIR-2	IR / identity	\$7.1
R4	Progress DET-003 and DET-001 through tuning first, on consequence rather than frequency. Baseline, allowlist, measure alert volume, then ship.	PIR-1	Detection engineering	\$11.1
R5	Then DET-004, which has the broadest evidence base (38 of 60 cases) and the noisiest false-positive profile — it needs the most tuning runway.	PIR-1	Detection engineering	\$11.1
R6	Repair the risk scorer before severity is used to prioritise anything. With 51 of 60 cases critical, the field ranks nothing. Verify the Lab 6 patch actually applied — read the file back, do not trust the print.	PIR-2	CTI lead	\$7.4, \$7.5
R7	Do not publish the STIX bundle. Fix deduplication and object_refs, validate against the OASIS schema, and keep the training-do-not-touch-live label through any refactor.	PIR-3	CTI lead	\$9.2, \$9.4
R8	Do not treat the 60-case cluster as a campaign, and do not block indicators estate-wide on the strength of it. PIR-3 is unanswered and should be recorded as such.	PIR-3	CTI lead	\$8.4
R9	Fix the analyst report template: technique names rather than tactic names, technique-specific defensive actions, and explicit mapping provenance on every report.	PIR-2	CTI lead	\$10.2

11.3 What the pipeline is legitimately good for today

It would be an over-correction to conclude the project produced nothing usable. Three outputs stand up on their own:

- **The extraction and graphing layers are sound.** Seventy-two indicators extracted, 302 relationships built, zero errors. Neither depends on the classifier.
- **The detection backlog is genuinely useful.** Five tickets with log sources, fields, expected false positives and a test method each — derived from validated mappings, not model output. That structure is what a detection engineer needs, and it survives every criticism in Section 6 because it never depended on the model.
- **The evaluation harness is the most valuable artefact in the project.** It found a leak, an artefact, a contamination bug and a generalisation collapse in a model that reported 0.90. A harness that can embarrass its own model is worth more than a model that scores well.

12. Limitations and Future Work

12.1 Limitations

Table 22. *Limitations, ordered by how much they constrain the conclusions in this report.*

#	Limitation	Consequence
L1	The corpus is synthetic and template-generated.	No metric in this report predicts performance on real vendor CTI reporting. Admiralty credibility 6 (§3.1) is the ceiling on every number here.
L2	Ten techniques are modelled; ATT&CK Enterprise has hundreds.	An unseen technique is silently impossible rather than merely wrong. The model cannot be wrong about T1078 — it can only be silent.
L3	The classifier memorises templates.	0.273 mean micro F1 on unseen phrasing; 0/5 on novel wording. The 0.900 headline measures memorisation.
L4	The vectoriser is fit before the split.	Train/test contamination. Modest for TF-IDF but real, and it inflates the reported figure.
L5	Techniques never appear alone — five fixed label bundles.	The model has no signal with which to separate co-occurring techniques, and co-occurrence in this corpus carries no information (§7.2).
L6	Risk scoring has five reachable outputs; 85% of cases are critical.	Severity ranks nothing. All five detection tickets came out P1 as a direct result.
L7	Confidence fields are constants, not computations.	Every case reads high, every indicator reads medium. A field that never varies conveys nothing but invites false trust.
L8	Clustering runs connected components over a graph containing taxonomy hubs.	One cluster of 60 cases. Corrected, seven — and still unusable, because only 15 indicators exist (§8.3).
L9	The STIX bundle is duplicated and relationship-free.	Not shareable. 362 objects, 85 ids, 0 relationships, and object_refs missing from every report SDO.
L10	Lab 8 external feeds were not authorised and not run.	No indicator in this report is corroborated by any external source. Nothing is cross-checked against anything.
L11	The Lab 6 sector tuning did not take effect.	The sector-weighted scoring described in §7.5 is unverified and its scores are the untuned defaults.
L12	Technique frequency reflects the generator's template mix.	Prioritising T1059 and T1041 because they are frequent here imports the generator's arbitrary weights into a real detection roadmap. Stated in §7.1 and repeated because it is the easiest error to make.
L13	The Lab 10 comparison used five of ten technique descriptions.	The embedding recommender could not reach T1027, T1041, T1053, T1090 or T1105. The comparison in §12.2 covers a five-label subset.

12.2 The most promising direction: semantic matching over supervised training

Lab 10 encoded each report and each of five ATT&CK technique descriptions with `all-MiniLM-L6-v2` and ranked techniques by cosine similarity. The model is zero-shot: it has never seen this corpus and cannot possibly have memorised a template.

```

Lab 10 — Embedding recommender

!pip install -q sentence-transformers
from sentence_transformers import SentenceTransformer, util
from threatfusion.ai.features import load_reports # importable now that we're in the project dir

model = SentenceTransformer('all-MiniLM-L6-v2')
technique_desc = {
    'T1566': 'phishing email to gain initial access',
    'T1059': 'command and scripting interpreter execution',
    'T1071': 'application layer protocol command and control',
    'T1003': 'operating system credential dumping',
    'T1486': 'data encrypted for impact (ransomware)',
    # fill in the remaining techniques
}
labels = list(technique_desc)
label_emb = model.encode(list(technique_desc.values()), convert_to_tensor=True)
for r in load_reports()[1:5]:
    emb = model.encode(r['title'] + ' ' + r['text'], convert_to_tensor=True)
    sims = util.cos_sim(emb, label_emb)[0]
    top = sorted(zip(labels, sims.tolist()), key=lambda x: -x[1])[1:3]
    print(r['id'], [(t, round(s, 2)) for t, s in top])

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
modules.json: 100% ██████████ 349/349 [00:00<00:00, 30.3kB/s]

config_sentence_transformers.json: 100% ██████████ 116/116 [00:00<00:00, 8.78kB/s]

README.md: 100% ██████████ 10.5k/10.5k [00:00<00:00, 847kB/s]

sentence_bert_config.json: 100% ██████████ 53.0/53.0 [00:00<00:00, 4.77kB/s]

```

Figure 23. The Lab 10 embedding recommender. Five technique descriptions are defined and the remaining five are left as a TODO — a limitation carried into the comparison below.

```

... Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
modules.json: 100% ██████████ 349/349 [00:00<00:00, 30.3kB/s]

config_sentence_transformers.json: 100% ██████████ 116/116 [00:00<00:00, 8.78kB/s]

README.md: 100% ██████████ 10.5k/10.5k [00:00<00:00, 847kB/s]

sentence_bert_config.json: 100% ██████████ 53.0/53.0 [00:00<00:00, 4.77kB/s]

config.json: 100% ██████████ 612/612 [00:00<00:00, 30.7kB/s]

model.safetensors: reconstructing file: 100% ██████████ 90.9MB / 90.9MB, 8.81MB/s

model.safetensors: downloading bytes: ██████████ 85.0MB, 8.14MB/s

Loading weights: 100% ██████████ 103/103 [00:00<00:00, 4643.81it/s]

tokenizer_config.json: 100% ██████████ 350/350 [00:00<00:00, 20.6kB/s]

vocab.bpe: 100% ██████████ 232k/232k [00:00<00:00, 9.02MB/s]

tokenizer.json: 100% ██████████ 466k/466k [00:00<00:00, 23.5MB/s]

special_tokens_map.json: 100% ██████████ 112/112 [00:00<00:00, 8.42kB/s]

config.json: 100% ██████████ 190/190 [00:00<00:00, 15.9kB/s]

LAB-BRAVO-000 [ ('T1003', 0.32), ('T1059', 0.23), ('T1071', 0.23) ]
LAB-CHARLIE-001 [ ('T1486', 0.64), ('T1003', 0.26), ('T1566', 0.16) ]
LAB-CHARLIE-002 [ ('T1486', 0.64), ('T1003', 0.23), ('T1566', 0.16) ]
LAB-CHARLIE-003 [ ('T1486', 0.62), ('T1003', 0.26), ('T1566', 0.17) ]
LAB-BRAVO-004 [ ('T1003', 0.39), ('T1059', 0.26), ('T1071', 0.25) ]

```

Figure 24. Embedding recommender output. The top-ranked technique falls inside the validated label set on all five sampled cases, and T1486 is ranked first with high separation on all three CHARLIE ransomware cases.

Table 23. Zero-shot embedding recommender versus the trained classifier. The recommender was never trained on this corpus.

Case	Validated labels	Embedding top-3 (cosine)	Top-1 valid?
LAB-BRAVO-000	T1003, T1027, T1059, T1090	T1003 (0.32) · T1059 (0.23) · T1071 (0.23)	YES
LAB-CHARLIE-001	T1041, T1071, T1486	T1486 (0.64) · T1003 (0.26) · T1566 (0.16)	YES
LAB-CHARLIE-002	T1041, T1071, T1486	T1486 (0.64) · T1003 (0.23) · T1566 (0.16)	YES
LAB-CHARLIE-003	T1041, T1071, T1486	T1486 (0.62) · T1003 (0.26) · T1566 (0.17)	YES
LAB-BRAVO-004	T1003, T1027, T1059, T1090	T1003 (0.39) · T1059 (0.26) · T1071 (0.25)	YES

Read the result carefully, in both directions.

- **In its favour:** top-1 lands inside the validated set on 5 of 5 cases. On the three CHARLIE ransomware reports it ranks T1486 first at 0.62–0.64, roughly 2.5× the next candidate — clean separation. On the BRAVO credential-access reports it puts T1003 first and T1059 second, recovering *both* of the labels it was capable of reaching. Meanwhile the trained classifier scores 0/5 on novel phrasing. A model that never saw the data beats the one trained on it.
- **Against it:** only five of ten technique descriptions were defined — the `# fill in the remaining techniques` comment was left unresolved — so T1027, T1041, T1053, T1090 and T1105 were unreachable by construction. LAB-BRAVO-000's true label set has four techniques and only two were in

the recommender's vocabulary. The comparison covers a five-label subset, and the 5/5 top-1 figure must be read with that ceiling in view.

- **A caution on the numbers:** cosine similarities of 0.16–0.64 are ranking signals, not calibrated confidences. A 0.64 does not mean 64% likely. Any production use needs a threshold established against a labelled set, not an intuition about what a cosine "feels" like.

Even with the caveats, this is the strongest evidence in the project. It is the cleanest possible demonstration that the 0.900 measures memorisation: remove the memorisation opportunity entirely, and performance goes *up*. The obvious next experiment is to complete `technique_desc` for all ten techniques and re-run the full comparison — classifier, recommender, and analyst judgement side by side, scored the same way on the same novel-phrasing probes.

12.3 Future work

Table 24. Future work, ordered so that the items which stop the pipeline from lying come before the items which make it cleverer.

Priority	Work item	Rationale
1 · Now	Fix the evaluation harness as a single change: drop title from features, fit the vectoriser on train only, stratify the split, and report leave-one-template-out alongside the random split on every run.	Four line-level edits that together make the reported number mean something. Expect it to fall — that is success.
2 · Now	Verify the Lab 6 scoring patch by reading the file back and asserting the replacement occurred, then re-run and diff the score table.	The current sector tuning is unverified and probably never applied (§7.5).
3 · Next	Complete <code>technique_desc</code> for all ten techniques and re-run the Lab 10 comparison as a fair ten-label evaluation against the same novel-phrasing probes.	Turns the most promising result in the project into a defensible one.
4 · Next	Corpus work: varied wording per technique, techniques appearing alone, and negative examples.	The single change that creates real generalisation signal. Nothing else substitutes for it.
5 · Next	Add abstention. Let the model return "no confident mapping" rather than the base rate.	§6.5 shows the failure mode is confident nonsense. Silence would have been more useful than either wrong answer.
6 · Next	Emit <code>object_refs</code> and relationship SDOs; deduplicate to 85 objects; validate against the OASIS schema in CI.	The relationships are already computed. This is serialisation, not analysis (§9.2).
7 · Later	Replace connected components with rarity-weighted community detection over the case↔IOC projection, excluding taxonomy nodes.	Removes the category error in §8.3 and makes clustering meaningful once the corpus can support it.
8 · Later	Hybrid architecture: embedding recommender for recall, classifier for precision on known phrasing, analyst validation as the gate before anything reaches a defensive decision.	Plays each component to its strength and keeps the human where the accountability is.
9 · Later	Compute confidence and TLP rather than hardcoding them; carry both into the analyst report and the STIX bundle.	§7.3, §10.2. A constant confidence field is worse than no confidence field.

12.4 Closing assessment

The pipeline runs end to end and produces every required artefact. The extraction, graphing and reporting layers work. The detection backlog is usable. But the two headline results this project could have claimed — a 0.90 F1 classifier and a 60-case campaign — are both artefacts, and the work of this report was establishing that rather than reporting them.

That is the outcome to defend. *a model that scores well on data it has memorised has told you about your data, not about your model.*

Appendix A · Evidence Portfolio Index

A.1 Submission checklist

Table 25. Final submission checklist from the assessment rubric, with the status and location of each item.

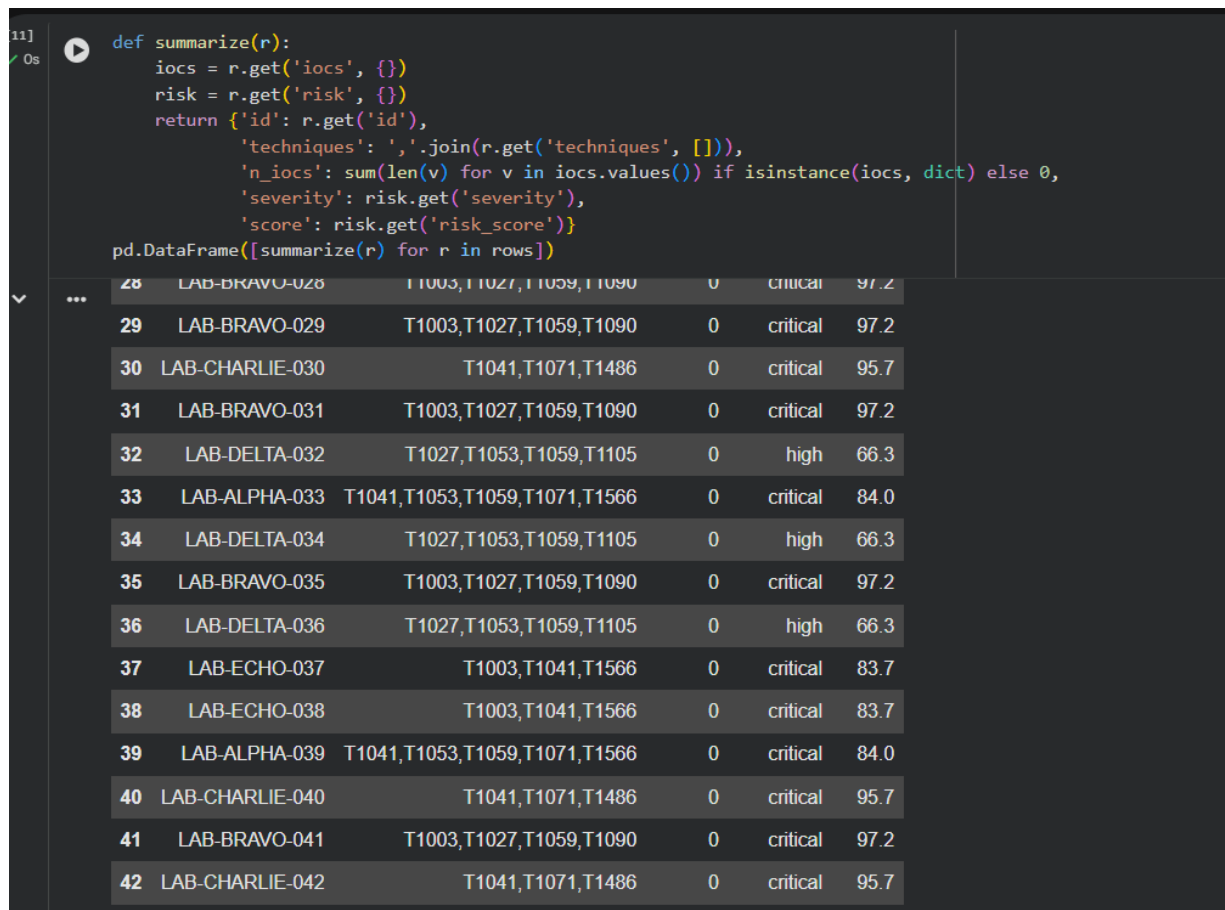
Required item	Status	Location / note
Project folder or repo — no .venv, no API keys	COMPLETE	Colab run: no .venv exists to exclude. No API key was ever present in the environment.
README with exact setup and run instructions	COMPLETE	Course README retained; add one line noting the Colab path (mount Drive → pip install -r requirements.txt → python -m threatfusion_ai.cli run-pipeline).
data/processed/enriched_cases.jsonl sample	COMPLETE	60 records. Structure shown at Figure 2.
outputs/model_evaluation.txt	COMPLETE	Macro F1 0.9000. Interpreted honestly in §6.
outputs/stix/threatfusion_bundle.json	COMPLETE	362 objects. Defects documented in §9.2 — not to be published.
outputs/graphs/threat_knowledge_graph.graphml (or .json)	COMPLETE	Both formats present. 85 nodes, 302 edges.
At least three analyst reports	COMPLETE	Five generated. §10.
Capstone report using the provided template	COMPLETE	This document — 12 sections plus this appendix.
Evidence portfolio with screenshots and reflections	COMPLETE	34 figures indexed below; the notebook itself is the lab log, run top to bottom.
Model Evaluation Report (20%)	COMPLETE	§5 and §6, with the Lab 4 evidence pack and four CSVs.
CTI Product Pack (25%)	COMPLETE	§2 PIRs, §3 source matrix, §8 graph, §9 STIX bundle, §10 reports.
Capstone defence (20%)	PENDING	Ten-minute demo. §12.4 is the closing argument; §6 is the core of the defence.

A.2 Open items carried forward

- **Verify the Lab 6 scoring patch** (§7.5) — read the file back and diff; do not trust the print statement.
- **Replace the PIRs in §2 with the Lab 2 originals** if they differ. The three recorded here were reconstructed from the questions the pipeline outputs actually answer.
- **Complete** `technique_desc` for all ten techniques (§12.2) before the Lab 10 comparison is quoted as a ten-label result.
- **Fix the** `n_iocs` inspection cell (§3.2) so the corpus review stops reporting a silent zero.

A.3 Figure index

Every figure is a screenshot of the live notebook run or a chart derived from the artefacts in the submission. Figures 26–29 are the full 60-case ledger, retained as evidence for the risk-scoring analysis in §7.4 and the no-op finding in §7.5.



```

11] def summarize(r):
    iocs = r.get('iocs', {})
    risk = r.get('risk', {})
    return {'id': r.get('id'),
            'techniques': ', '.join(r.get('techniques', [])),
            'n_iocs': sum(len(v) for v in iocs.values()) if isinstance(iocs, dict) else 0,
            'severity': risk.get('severity'),
            'score': risk.get('risk_score')}
pd.DataFrame([summarize(r) for r in rows])

```

28	LAB-BRAVO-028	T1003,T1027,T1059,T1090	0	critical	97.2
29	LAB-BRAVO-029	T1003,T1027,T1059,T1090	0	critical	97.2
30	LAB-CHARLIE-030	T1041,T1071,T1486	0	critical	95.7
31	LAB-BRAVO-031	T1003,T1027,T1059,T1090	0	critical	97.2
32	LAB-DELTA-032	T1027,T1053,T1059,T1105	0	high	66.3
33	LAB-ALPHA-033	T1041,T1053,T1059,T1071,T1566	0	critical	84.0
34	LAB-DELTA-034	T1027,T1053,T1059,T1105	0	high	66.3
35	LAB-BRAVO-035	T1003,T1027,T1059,T1090	0	critical	97.2
36	LAB-DELTA-036	T1027,T1053,T1059,T1105	0	high	66.3
37	LAB-ECHO-037	T1003,T1041,T1566	0	critical	83.7
38	LAB-ECHO-038	T1003,T1041,T1566	0	critical	83.7
39	LAB-ALPHA-039	T1041,T1053,T1059,T1071,T1566	0	critical	84.0
40	LAB-CHARLIE-040	T1041,T1071,T1486	0	critical	95.7
41	LAB-BRAVO-041	T1003,T1027,T1059,T1090	0	critical	97.2
42	LAB-CHARLIE-042	T1041,T1071,T1486	0	critical	95.7

Figure 25. Full case ledger, cases 28–42 (cell 11, before the Lab 6 patch). The *n_iocs* column reads 0 for every case — an artefact of the inspection cell, explained in §3.2.

...	43	LAB-BRAVO-043	T1003,T1027,T1059,T1090	0	critical	97.2
	44	LAB-CHARLIE-044	T1041,T1071,T1486	0	critical	95.7
	45	LAB-ALPHA-045	T1041,T1053,T1059,T1071,T1566	0	critical	84.0
	46	LAB-DELTA-046	T1027,T1053,T1059,T1105	0	high	66.3
	47	LAB-BRAVO-047	T1003,T1027,T1059,T1090	0	critical	97.2
	48	LAB-ALPHA-048	T1041,T1053,T1059,T1071,T1566	0	critical	84.0
	49	LAB-ALPHA-049	T1041,T1053,T1059,T1071,T1566	0	critical	84.0
	50	LAB-ECHO-050	T1003,T1041,T1566	0	critical	83.7
	51	LAB-CHARLIE-051	T1041,T1071,T1486	0	critical	95.7
	52	LAB-BRAVO-052	T1003,T1027,T1059,T1090	0	critical	97.2
	53	LAB-CHARLIE-053	T1041,T1071,T1486	0	critical	95.7
	54	LAB-DELTA-054	T1027,T1053,T1059,T1105	0	high	66.3
	55	LAB-DELTA-055	T1027,T1053,T1059,T1105	0	high	66.3
	56	LAB-DELTA-056	T1027,T1053,T1059,T1105	0	high	66.3
	57	LAB-ALPHA-057	T1041,T1053,T1059,T1071,T1566	0	critical	84.0
	58	LAB-BRAVO-058	T1003,T1027,T1059,T1090	0	critical	97.2
	59	LAB-ECHO-059	T1003,T1041,T1566	0	critical	83.7

Figure 26. Full case ledger, cases 43–59 (cell 11, before the Lab 6 patch). Compare these scores against Figures 28–29.

3	LAB-CHARLIE-003	critical	95.7
4	LAB-BRAVO-004	critical	97.2
5	LAB-CHARLIE-005	critical	95.7
6	LAB-DELTA-006	high	66.3
7	LAB-ALPHA-007	critical	84.0
8	LAB-BRAVO-008	critical	97.2
9	LAB-CHARLIE-009	critical	95.7
10	LAB-CHARLIE-010	critical	95.7
11	LAB-ALPHA-011	critical	84.0
12	LAB-BRAVO-012	critical	97.2
13	LAB-ECHO-013	critical	83.7
14	LAB-ECHO-014	critical	83.7
15	LAB-ALPHA-015	critical	84.0
16	LAB-BRAVO-016	critical	97.2
17	LAB-CHARLIE-017	critical	95.7
18	LAB-ECHO-018	critical	83.7
19	LAB-DELTA-019	high	66.3
20	LAB-ECHO-020	critical	83.7
21	LAB-BRAVO-021	critical	97.2
22	LAB-BRAVO-022	critical	97.2

Figure 27. Risk scores after the Lab 6 patch, cases 3–22 (cell 19).

23	LAB-ALPHA-023	critical	84.0
24	LAB-CHARLIE-024	critical	95.7
25	LAB-ALPHA-025	critical	84.0
26	LAB-BRAVO-026	critical	97.2
27	LAB-ALPHA-027	critical	84.0
28	LAB-BRAVO-028	critical	97.2
29	LAB-BRAVO-029	critical	97.2
30	LAB-CHARLIE-030	critical	95.7
31	LAB-BRAVO-031	critical	97.2
32	LAB-DELTA-032	high	66.3
33	LAB-ALPHA-033	critical	84.0
34	LAB-DELTA-034	high	66.3
35	LAB-BRAVO-035	critical	97.2
36	LAB-DELTA-036	high	66.3
37	LAB-ECHO-037	critical	83.7
38	LAB-ECHO-038	critical	83.7
39	LAB-ALPHA-039	critical	84.0
40	LAB-CHARLIE-040	critical	95.7
41	LAB-BRAVO-041	critical	97.2
42	LAB-CHARLIE-042	critical	95.7
43	LAB-BRAVO-043	critical	97.2

Figure 28. Risk scores after the Lab 6 patch, cases 23–43 (cell 19).

44	LAB-CHARLIE-044	critical	95.7
45	LAB-ALPHA-045	critical	84.0
46	LAB-DELTA-046	high	66.3
47	LAB-BRAVO-047	critical	97.2
48	LAB-ALPHA-048	critical	84.0
49	LAB-ALPHA-049	critical	84.0
50	LAB-ECHO-050	critical	83.7
51	LAB-CHARLIE-051	critical	95.7
52	LAB-BRAVO-052	critical	97.2
53	LAB-CHARLIE-053	critical	95.7
54	LAB-DELTA-054	high	66.3
55	LAB-DELTA-055	high	66.3
56	LAB-DELTA-056	high	66.3
57	LAB-ALPHA-057	critical	84.0
58	LAB-BRAVO-058	critical	97.2
59	LAB-ECHO-059	critical	83.7

Figure 29. Risk scores after the Lab 6 patch, cases 44–59 (cell 19). Every value is identical to the pre-patch run in Figure 27 — the evidence for the no-op finding in §7.5.

Table 26. Figure index. Screenshots are from the live Colab notebook run of 17 July 2026; charts are derived from the artefacts in the submission.

Fig.	Description	Source
1	The size of the illusion.	Derived chart
2	Enriched case record for LAB-BRAVO-000.	Notebook screenshot
3	ThreatFusion AI module data flow.	Derived chart
4	Environment verification and Drive mount.	Notebook screenshot
5	Dependency install, the timezone-aware patch to stix_exporter.	Notebook screenshot
6	run-pipeline: 60 cases processed in 10 s, producing the knowledge graph, the STIX bundle, five analyst reports and the campaign clustering.	Notebook screenshot
7	Classifier training output.	Notebook screenshot
8	The Lab 4 error-analysis harness.	Notebook screenshot

Fig.	Description	Source
9	Lab 4 diagnostics.	Notebook screenshot
10	Technique frequency across the 60-case corpus.	Derived chart
11	Risk score by template family.	Derived chart
12	The Lab 6 scoring patch and the pipeline re-run.	Notebook screenshot
13	Knowledge graph load and node-type inspection: 85 nodes, 302 edges, with case, attack-technique and IOC node types.	Notebook screenshot
14	Static view of the knowledge graph, coloured by node type: cases in orange, ATT&CK techniques in purple, indicators in grey.	Notebook screenshot
15	The same graph rendered interactively with pyvis for hands-on inspection.	Notebook screenshot
16	Campaign clustering output: a single cluster spanning every case in the corpus.	Notebook screenshot
17	Cluster decomposition.	Derived chart
18	STIX bundle inspection.	Notebook screenshot
19	Emitted objects versus distinct identifiers.	Derived chart
20	LAB-CHARLIE-003 rendered.	Notebook screenshot
21	The Lab 11 backlog generator.	Notebook screenshot
22	Backlog generation: 60 cases read, mapping source validated, 10 techniques with evidence, 5 tickets written.	Notebook screenshot
23	The Lab 10 embedding recommender.	Notebook screenshot
24	Embedding recommender output.	Notebook screenshot
25	Full case ledger, cases 28–42 (cell 11, before the Lab 6 patch).	Notebook screenshot
26	Full case ledger, cases 43–59 (cell 11, before the Lab 6 patch).	Notebook screenshot
27	Risk scores after the Lab 6 patch, cases 3–22 (cell 19).	Notebook screenshot
28	Risk scores after the Lab 6 patch, cases 23–43 (cell 19).	Notebook screenshot
29	Risk scores after the Lab 6 patch, cases 44–59 (cell 19).	Notebook screenshot

For authorised defensive training only. AICS-107 · AI-Driven Cybersecurity and Digital Forensics Fellowship.